



UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA
INGENIERÍA EN SISTEMAS DE INFORMACIÓN

ANÁLISIS DE DATOS EMPRESARIALES

Trabajo Práctico Final

Grupo 2 - Escenario Discográfica

Profesor:

- Ing. Andrés Pablo **Fantin**.

Alumnos:

- Aldo Omar **Andres**.
- Sixto Feliciano **Arrejin**.
- Agustín Nicolás **Bravo Pérez**.
- André Leandro **San Lorenzo**.

30 de Abril de 2025

Índice

Consigna	1
1. Carga Inicial del Data Warehouse (DW)	1
2. Consideraciones de Reprocesamiento	2
3. Carga Incremental y Actualización del DW	2
4. Cambios en Dimensiones con SCD	2
5. (Opcional) Aplicación de DQS y DMS	3
Entrega Final	3
Desarrollo	4
Carga Inicial Masiva	4
Consideraciones	4
Cambios en el Diseño Lógico del Data Warehouse	4
Fuentes de Datos	4
Configuración	5
Truncar Data Warehouse	6
Insertar Registros Comodines	7
Generar DimTiempo	8
ETL Dimensiones Hoja	9
DimUbicacion	10
DimInstrumento	12
DimCategoria	15
DimManager	15
ETL DimTienda y DimBanda	17
DimTienda	17
DimBanda	18
ETL DimArtista y DimDisco	22
DimArtista	22
DimDisco	23
Asignar representantes en DimBanda	28
ETL Hechos	29
HechoVentas	29
HechoProducciones	34
Carga Incremental	36
ETL Incremento HechoVentas	37
ETL Incremento HechoProducciones	42
Slowly Changing Dimensions	43
Incrementar DimArtista (SCD)	43
Incrementar DimBanda (SCD)	45
Incrementar DimDiscos	48
Data Quality Services	50
Master Data Services	52

Consigna

El objetivo de este trabajo es poner en práctica los conceptos fundamentales de integración de datos, modelado multidimensional y carga de un Data Warehouse (DW), incluyendo consideraciones de integridad, historización y actualización incremental. Se valorará además el uso de herramientas complementarias como Data Quality Services (DQS) y Data Management Services (DMS).

1. Carga Inicial del Data Warehouse (DW)

Una vez finalizado el diseño del modelo multidimensional (hechos y dimensiones) en el TP anterior, se procederá a implementar el proceso de carga inicial masiva desde los datos operacionales o archivos fuentes al Data Warehouse.

Tareas requeridas

1. *Obtención y preparación de datos fuentes:*
 - *Cada grupo deberá proveerse de los archivos de origen (CSV, Excel o exportaciones SQL) necesarios, con asistencia de la cátedra.*
 - *Validar que los datos contengan claves y atributos completos y consistentes (sin nulos en campos clave).*
2. *Desarrollo de un proceso ETL inicial:*
 - *Utilizar una herramienta como SQL Server Integration Services (SSIS) o scripts SQL personalizados.*
 - *Implementar un flujo de carga en el siguiente orden:*
 - *Staging Area: importar los archivos en tablas intermedias (staging) para validación previa.*
 - *Transformaciones: limpieza, conversión de tipos, normalización, eliminación de duplicados.*
 - *Carga de dimensiones: asegurarse de que todas las claves de las dimensiones estén cargadas antes de procesar la tabla de hechos.*
 - *Carga de la tabla de hechos: asegurar que todas las claves externas estén resueltas.*
3. *Controles de integridad:*
 - *Validar la integridad referencial entre hechos y dimensiones (FOREIGN KEY).*
 - *Aplicar lógica de sustitución de claves en caso de registros desconocidos o faltantes (e.g., surrogate key “-1” para “Unknown”).*
4. *Diseño de carga repetible (idempotente):*
 - *El proceso debe poder ejecutarse múltiples veces sin generar errores ni datos duplicados.*
 - *Estrategias sugeridas:*
 - *Truncado de tablas antes de carga (TRUNCATE TABLE).*
 - *Detección de duplicados mediante claves naturales.*

- *Uso de transacciones para asegurar atomicidad.*
 - *Control de errores y registros rechazados (logging).*
5. *Optimización de performance (bonus):*
- *Aplicar índices temporales en staging.*
 - *Procesar en bloques para grandes volúmenes.*
 - *Desactivar temporalmente constraints o triggers si fuera necesario (y reactivarlos luego).*

2. Consideraciones de Reprocesamiento

El proceso ETL debe estar preparado para ser ejecutado más de una vez sin generar errores ni duplicaciones.

Recomendaciones:

- *Uso de TRUNCATE o MERGE según el caso.*
- *Validaciones previas para evitar claves duplicadas.*
- *Carga en tablas staging antes de inserciones definitivas.*

3. Carga Incremental y Actualización del DW

Ha transcurrido un mes desde la carga inicial. Ahora deben incorporar nuevos datos provenientes del sistema OLTP sin borrar ni rehacer todo el DW.

Tareas a realizar:

- *Diseñar un proceso incremental que detecte e inserte solo los nuevos registros o los modificados.*
- *Explicar y demostrar cómo modificarían el flujo de ETL existente para soportar estas cargas periódicas.*
- *Implementar controles de cambios y fechas de actualización (LastUpdatedDate, EffectiveDate, etc.).*

Aspectos clave:

- *Separación entre datos históricos y actuales.*
- *Optimización del tiempo de ejecución.*
- *Uso de flags, timestamps o hash para detección de cambios.*

4. Cambios en Dimensiones con SCD

Explique qué ocurre cuando un atributo relevante de una dimensión sufre modificaciones.

Contenido esperado:

- *Tipos de Slowly Changing Dimensions (SCD): Tipo 1, 2 y 3.*
- *Ejemplo implementado en SSIS: usar el componente Slowly Changing Dimension con datos reales.*
- *Mostrar con un query antes y después de una carga que genera una nueva versión del registro (si corresponde a SCD Tipo 2).*

5. (Opcional) Aplicación de DQS y DMS

Como parte opcional para obtener puntaje adicional:

Aplicar DQS (Data Quality Services):

- *Crear un knowledge base para estandarizar campos como nombre de clientes o códigos de ciudades.*
- *Integrar DQS en el flujo de carga para limpiar y enriquecer datos.*

Aplicar DMS (Data Management Services):

- *Registrar metadatos del DW: descripciones de campos, relaciones, responsables de mantenimiento.*
- *Usar plantillas o herramientas para documentar los activos de datos.*

Entrega Final

Debe subirse un archivo .zip que contenga:

- *Carpeta del proyecto ETL (archivo .sln, .dtsx, etc.).*
- *Enlaces a archivos fuente utilizados o copia de los mismos.*
- *Documento explicativo en formato PDF, detallando:*
- *Estrategia de carga inicial y de cargas incrementales.*
 - *Manejo de SCD y control de versiones.*
 - *Capturas de pantalla relevantes de los paquetes ETL y resultados.*
 - *Consideraciones de calidad de datos (si corresponde).*
 - *Fuentes utilizadas (incluyendo estructura y tamaño aproximado).*
 - *Diagrama del flujo ETL.*
 - *Descripción de las principales transformaciones.*
 - *Capturas de pantalla del proceso en ejecución.*

Desarrollo

El objetivo de este documento es explicar detalladamente el proceso que llevó a cabo el equipo para poner en práctica los conceptos vistos en clases y crear el ETL para el DW de la discográfica (nuestro escenario). A lo largo del mismo se irá explicando todo el desarrollo llevado a cabo.

Para el desarrollo de las actividades propuestas decidimos dividir las en tres paquetes diferentes:

1. **Carga inicial:** involucra a todas las dimensiones y a todos los hechos definidos en el DW.
2. **Carga incremental:** foco en los hechos y solo algunas dimensiones.
3. **Slow Changing Dimensions:** foco en las dimensiones DimBanda y DimArtista.

Carga Inicial

Consideraciones

Cambios en el Diseño Lógico del Data Warehouse

A medida que avanzamos el ETL, identificamos cambios a realizar en el script SQL que crea las tablas del Data Warehouse. La mayoría se deben a que no conocíamos las diferencias de MS SQL Server con respecto a otras bases de datos relacionales. Los listamos en esta sección:

- Agregar IDENTITY(1,1) a todas las claves primarias para que el motor de la base de datos les asigne un valor autoincremental.
- Cambiar todos los tipos de datos VARCHAR a NVARCHAR para que acepten caracteres unicode.
- Eliminar la columna DimInstrumento.familia por ser igual a tipo_instrumento.
- Modificación de varios tipos de datos para adecuarlos a los data sources. Principalmente alargamos los NVARCHAR a 255 para no truncar los datos.
- Agregamos las columnas DimBanda.desde y DimBanda.hasta para considerar SCD de tipo 2.

El nuevo esquema del DW está en este archivo: `CreateDiscografica.sql`.

Fuentes de Datos

En la siguiente planilla se indica la fuente de datos que utilizamos para cada columna del Data Warehouse: `ADE - TPI - Data Sources`.

Nuestra fuente de datos principal es **musicbrainz**, una base de datos relacionada a bandas y discos. En https://musicbrainz.org/doc/MusicBrainz_Database/Download se explica cómo reproducir musicbrainz en un PostgreSQL. Conviene informar que algunos integrantes tuvieron problemas para reproducir musicbrainz localmente, y entonces usaron como fuente archivos CSVs con los resultados de las queries SQL.

La mayoría de datos los obtenemos de musicbrainz. Los datos para las dimensiones de Tienda y Ubicación se obtienen de un dataset con **tiendas de McDonald's** ([McDonald's Stores Data](#)).

Finalmente, en el archivo `mock.ipynb` generamos tres archivos CSV con los datos de ventas, producciones, y managers. Se usa multiprocessing para generar rápidamente los datos de ventas. Esto requiere que la función paralelizada esté en un archivo aparte: `ventas.py`.

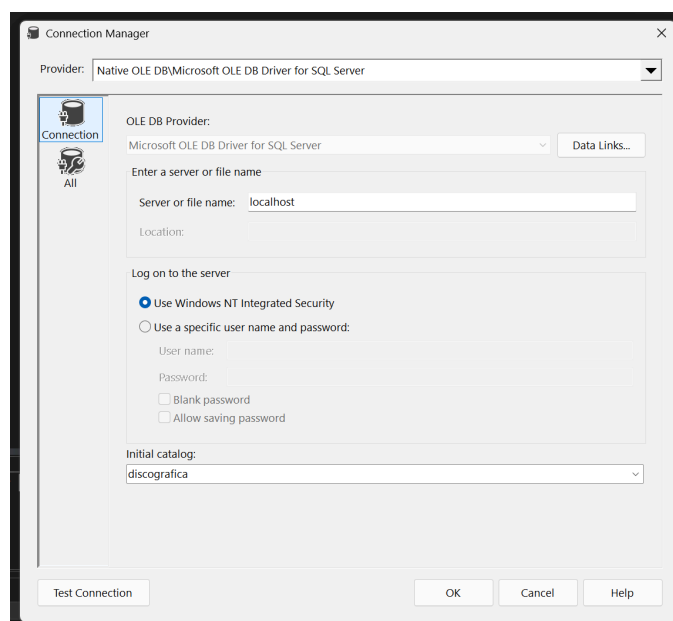
Se consulta la tabla `release_group` de `musicbrainz` para obtener los discos y relacionarlos con las ventas y producciones generadas. Similarmente, se obtienen las bandas de `artist` para relacionarlas con los managers generados. También se relacionan las tiendas de McDonald's con las ventas generadas. Se utiliza una semilla definida para que los datos generados sean siempre los mismos. Se agrega una columna `id_tienda` (valor entero autoincremental) al excel de `McDonaldsStores`, debido a que ese data source no tiene una clave primaria.

Se generan:

- 10.000.000 ventas, cada una asociada a una tienda y un disco.
- 2.162.075 producciones, cada una asociada a un disco distinto (relación 1 a 1).
- 92.894 managers, cada uno asociado a una banda distinta. El 25% de bandas quedan sin un manager asociado, para que en el ETL se tenga que asignar a un miembro de la banda como representante.

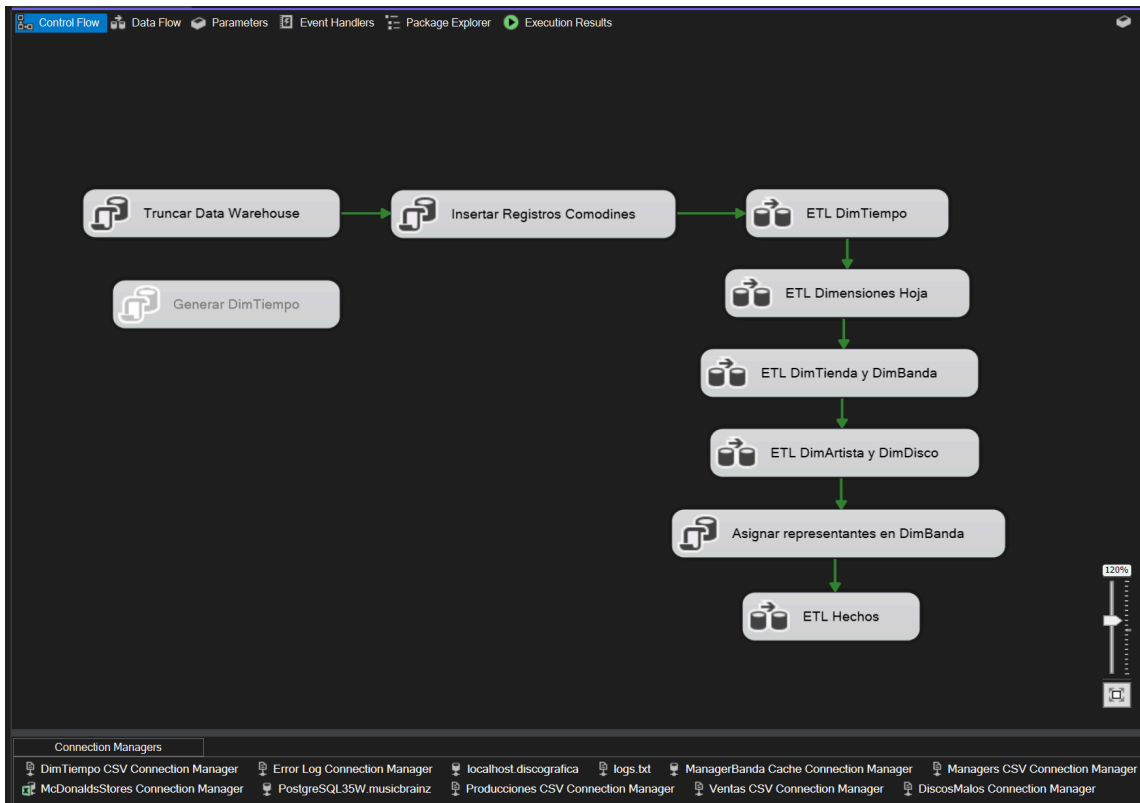
Configuración

Desarrollamos el ETL en un proyecto de Visual Studio utilizando la extensión de SQL Server Integration Services (SSIS), con el Data Warehouse previamente levantado en una base de datos SQL Server 2022. El script para crear las tablas de la base de datos *discografica* es el siguiente: `Discografica.sql`. En el proyecto de Visual Studio se configura una conexión para el Data Warehouse.



Conexión a la base de datos *discografica* del Data Warehouse.

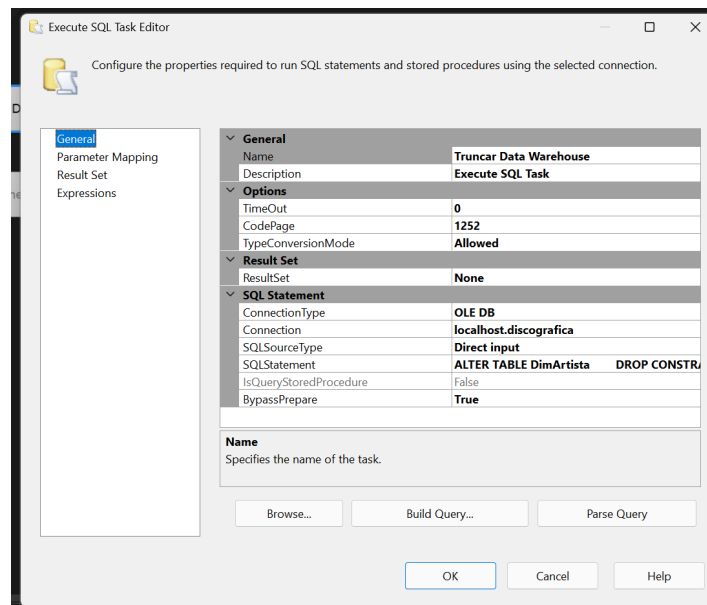
Se debe configurar una conexión para cada fuente de datos. Lo iremos haciendo en cada paso. El paquete de la carga inicial se ve así.



Control Flow de la carga inicial.

Ahora iremos viendo que hicimos en cada uno de estos Flujos de Datos y que consideraciones tuvimos en cuenta para ellos.

Truncar Data Warehouse



Configuración del Execute SQL Task llamado Trunca Data Warehouse.

Esa Task ejecuta el siguiente script SQL cargado en la propiedad SQLStatement, el cual elimina todos los datos de la base de datos: **DropDiscografica.sql**.

Esto permite ejecutar varias veces el ETL sin problemas, ideal para iterar durante el desarrollo. Ahora, para cargar las tablas del Data Warehouse, hay que tomar el diseño conceptual e ir desde afuera hacia adentro.

Insertar Registros Comodines

The screenshot shows the 'Execute SQL Task Editor' window. On the left is a sidebar with 'General' selected. The main area displays the configuration for a task named 'Insertar Registros Comodines'. The configuration is organized into sections: General, Options, Result Set, and SQL Statement. The 'SQL Statement' section contains a text area with the query '----- REGISTROS COMODINES PARA REGIST'. At the bottom are buttons for 'Browse...', 'Build Query...', 'Parse Query', 'OK', 'Cancel', and 'Help'.

General	
Name	Insertar Registros Comodines
Description	Execute SQL Task

Options	
TimeOut	0
CodePage	1252
TypeConversionMode	Allowed

Result Set	
ResultSet	None

SQL Statement	
ConnectionType	OLE DB
Connection	localhost.discografica
SQLSourceType	Direct input
SQLStatement	----- REGISTROS COMODINES PARA REGIST
IsQueryStoredProcedure	False
BypassPrepare	True

Name
Specifies the name of the task.

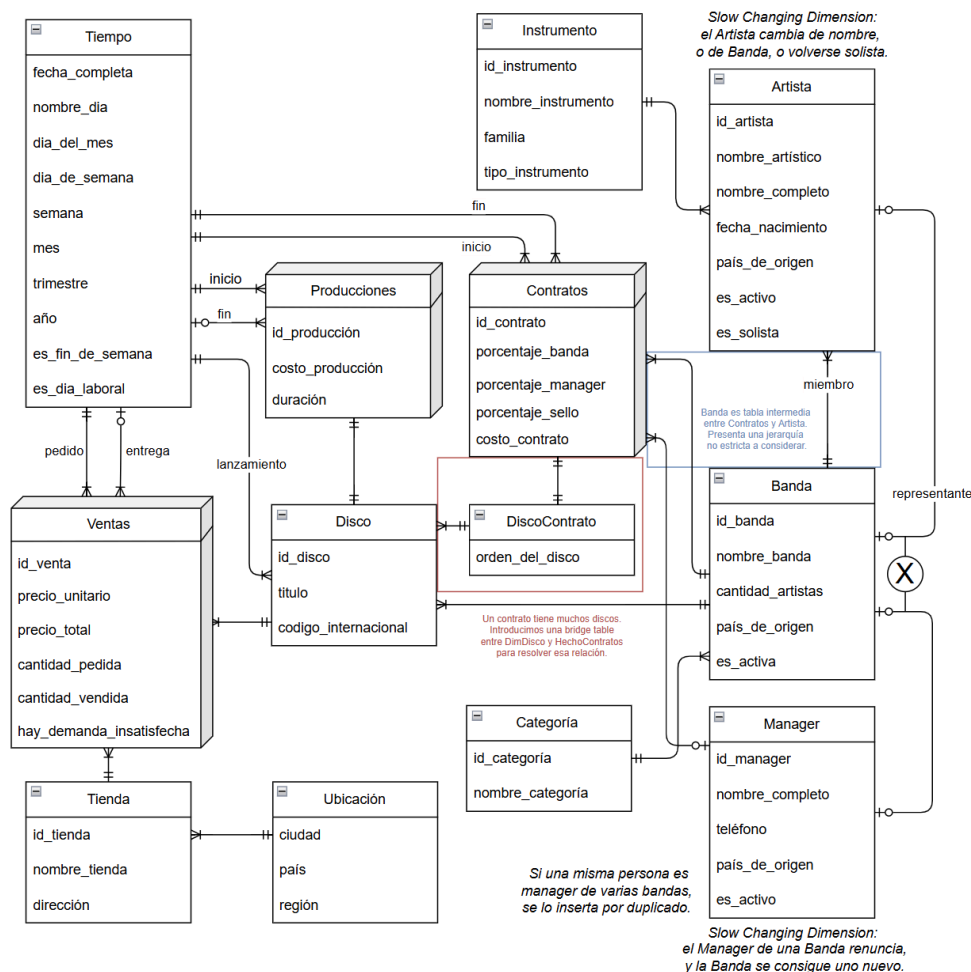
Browse... Build Query... Parse Query

OK Cancel Help

Configuración de Insertar Registros Comodines.

Esta tarea se encarga de insertar registros "comodín" o por defecto en aquellas dimensiones del Data Warehouse donde podría faltar información clave, como por ejemplo valores nulos en campos obligatorios. Sirve para evitar errores en la carga de datos o en futuras consultas, asegurando que, si un dato está incompleto o ausente, se le asigne un valor genérico como "unknown" o "desconocido" en lugar de dejar el campo vacío. De esta forma, el ETL puede continuar sin interrupciones y se mantiene la integridad referencial del modelo.

Ahora echemos un vistazo al Esquema del Data Warehouse (DW) para la discográfica, que será la base sobre la cual comenzaremos a realizar la carga de cada una de las tablas correspondientes. Este esquema representa la estructura dimensional definida en el diseño conceptual y lógico del proyecto, incluyendo tanto las dimensiones como las tablas de hechos.



Esquema del DW para la discografía (versión entregada en el TPI Parte 1).

Antes de comenzar con la carga de datos es importante tener claro el orden en que deben cargarse las tablas, ya que algunas dependen de otras mediante claves foráneas. El flujo de carga adecuado sigue el siguiente orden:

1. Dimensiones hoja: Tiempo, Ubicación, Categoría, Manager, Instrumento.
2. Dimensiones intermedias: Tienda, Banda (sin la FK dwid_representante porque todavía no se cargaron los artistas).
3. Dimensiones intermedias: Artista, Disco.
4. Cargar la columna dwid_representante en Banda.
5. Hechos: Ventas, Producciones.

Recordemos que Contratos y DiscoContrato se ignoran en el ETL para simplificar nuestro trabajo, dado que el escenario que nos tocó es más complejo de lo ideal.

Con este panorama claro, damos comienzo a la carga de datos, empezando por la Dimensión Tiempo, que es fundamental para permitir el análisis temporal de ventas y producciones.

Generar DimTiempo

Generamos los datos de la tabla DimTiempo utilizando el script compartido por el profesor en slack, adaptado a los campos que nosotros definimos.

El rango de fechas generado abarca desde el año 1888, que corresponde al inicio de los registros históricos de lanzamientos de discos según la base de datos de MusicBrainz, hasta el año 2045. Aunque existen algunos registros fuera de ese rango, los consideramos *potencialmente erróneos* y se dejó pendiente su validación con el profesor para decidir si deben ser cargados o descartados.

Para agilizar el proceso de desarrollo y facilitar pruebas iterativas, los datos generados se almacenaron en un archivo DimTiempo.csv, el cual es cargado directamente al Data Warehouse mediante un Data Flow Task en SSIS. Durante esta carga, se omite la columna dwid_tiempo ya que está definida como autoincremental (IDENTITY) en la base de datos.

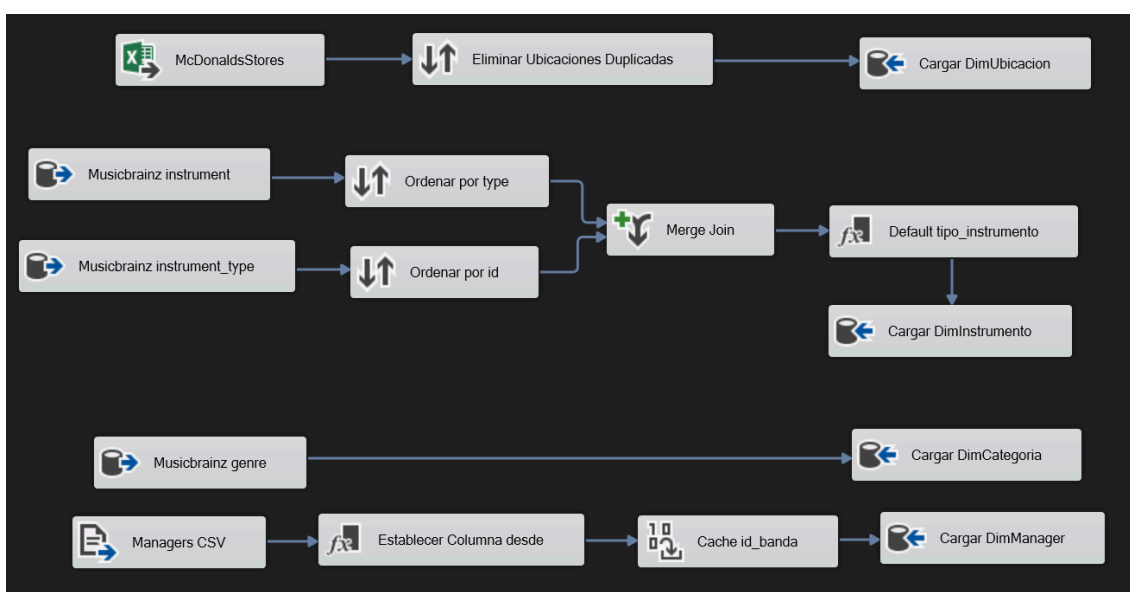


Data Flow del ETL DimTiempo.

Finalmente, se realizó una pequeña corrección técnica en el componente de carga, ajustando el tipo de dato de la columna nombre_dia a Unicode (DT_WSTR) para evitar errores de compatibilidad con caracteres especiales. Con esta dimensión cargada, el sistema queda listo para realizar análisis temporales robustos sobre las tablas de hechos.

Acá nos dimos cuenta que este cambio debíamos realizarlo en todos los campos de tiempo texto o string en todo el desarrollo del ETL y así lo hicimos.

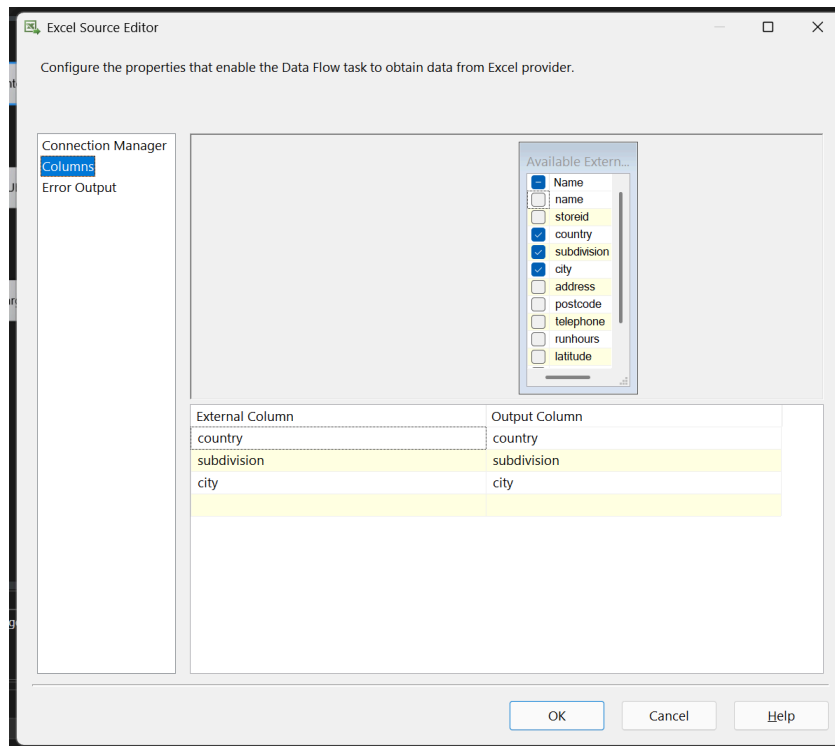
ETL Dimensiones Hoja



Data Flow del ETL Dimensiones Hoja.

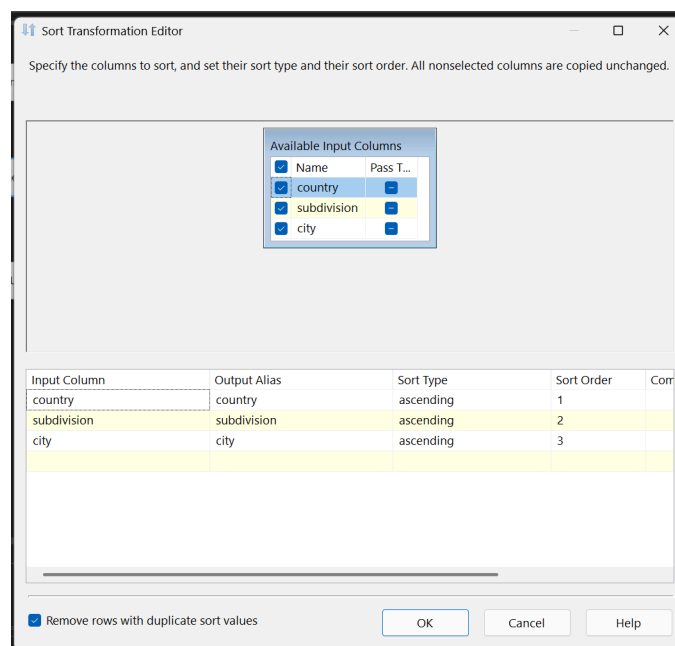
DimUbicacion

La dimensión DimUbicacion se construye a partir del archivo Excel con datos de tiendas McDonald's, que usamos como fuente externa. En este flujo de datos, se seleccionan únicamente las columnas relacionadas con la ubicación geográfica (como país, ciudad, dirección, etc.).



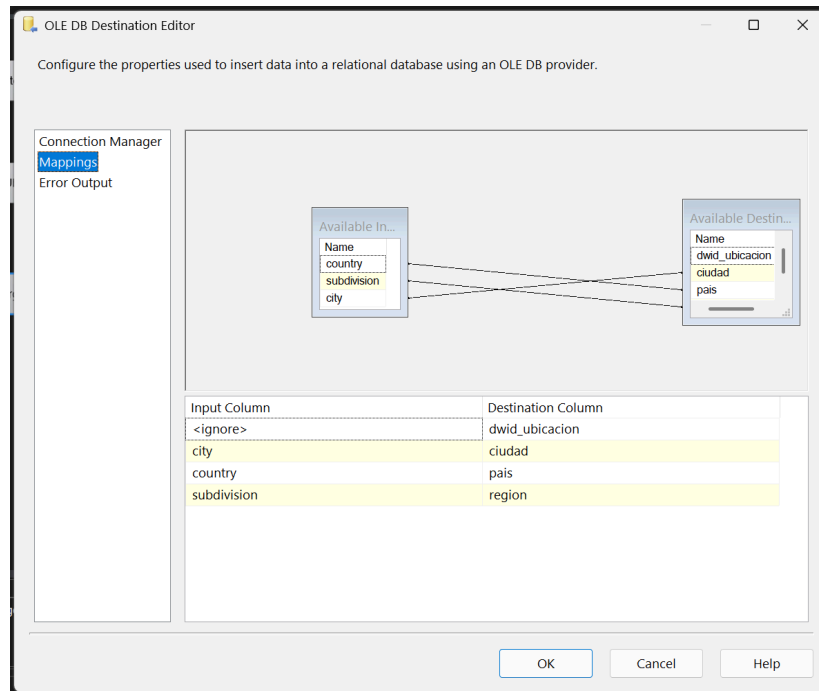
En Columns se definen las columnas a leer. Ahora solo interesan las de ubicación.

Antes de cargar los datos en el Data Warehouse, se eliminan los registros duplicados para evitar inconsistencias. Para esto, se utiliza un componente de ordenamiento que, además de ordenar, permite eliminar filas duplicadas marcando la opción correspondiente.



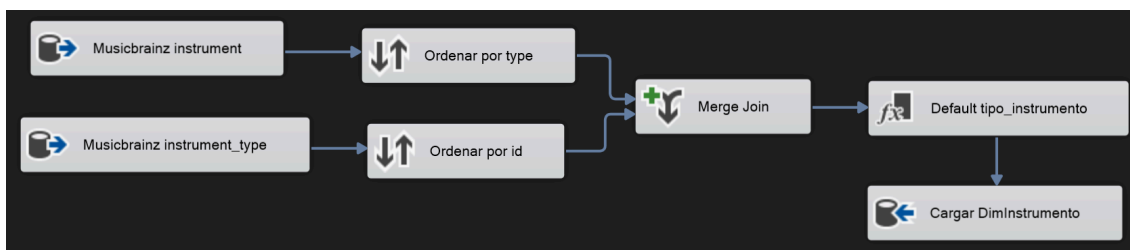
Configuración del componente Eliminar Ubicaciones Duplicadas.

Una vez depurados, los datos se cargan directamente en la tabla DimUbicacion. Este tipo de dimensiones, al no tener dependencias con otras tablas, se consideran "dimensiones hoja" y su carga suele ser directa y sin mayores complicaciones.



Mappings del componente Cargar DimUbicacion.

DimInstrumento



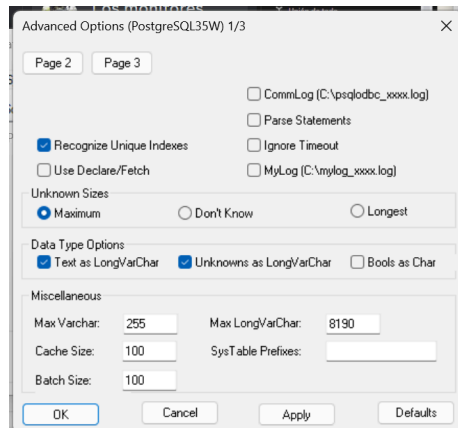
ETL para DimInstrumento.

La dimensión DimInstrumento se construye a partir de dos tablas de la base de datos de MusicBrainz: instrument y instrument_type. Para acceder a estos datos desde SSIS, se configuraron componentes ODBC Source, utilizando el driver PostgreSQL Unicode (x64). En caso de que este driver no esté instalado, es necesario descargarlo e instalarlo previamente.

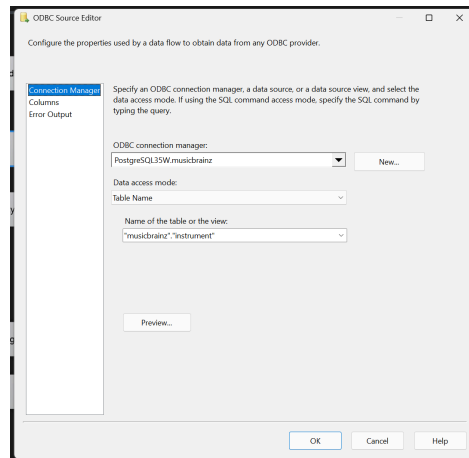
Del conjunto de datos original, se seleccionaron únicamente las columnas relevantes: id, name y type de la tabla instrument, y id y name de instrument_type. Ambos flujos se ordenaron (con Sort Transformations) para poder ser combinados mediante un Merge Join, ya que este tipo de componente requiere que las entradas estén previamente ordenadas.

Durante la transformación, los registros que no contienen información sobre el tipo de instrumento (NULL) se completan con el valor por defecto "unknown" para mantener la consistencia de los datos.

Finalmente, los datos resultantes se cargan en la tabla DimInstrumento. Al igual que otras dimensiones hoja, esta carga es relativamente directa una vez realizada la preparación y limpieza inicial.



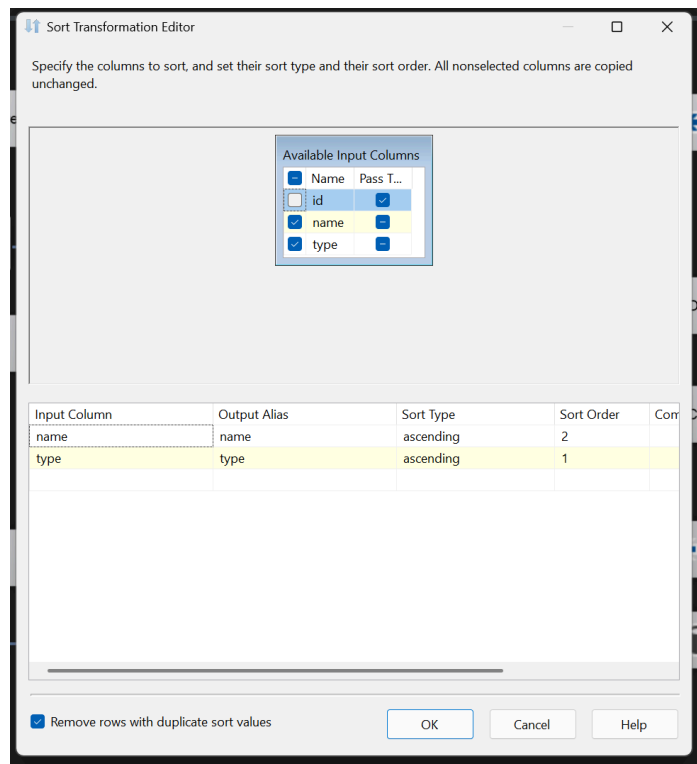
Configuración del ODBC Driver para PostgreSQL. Esto es afuera de SSIS.



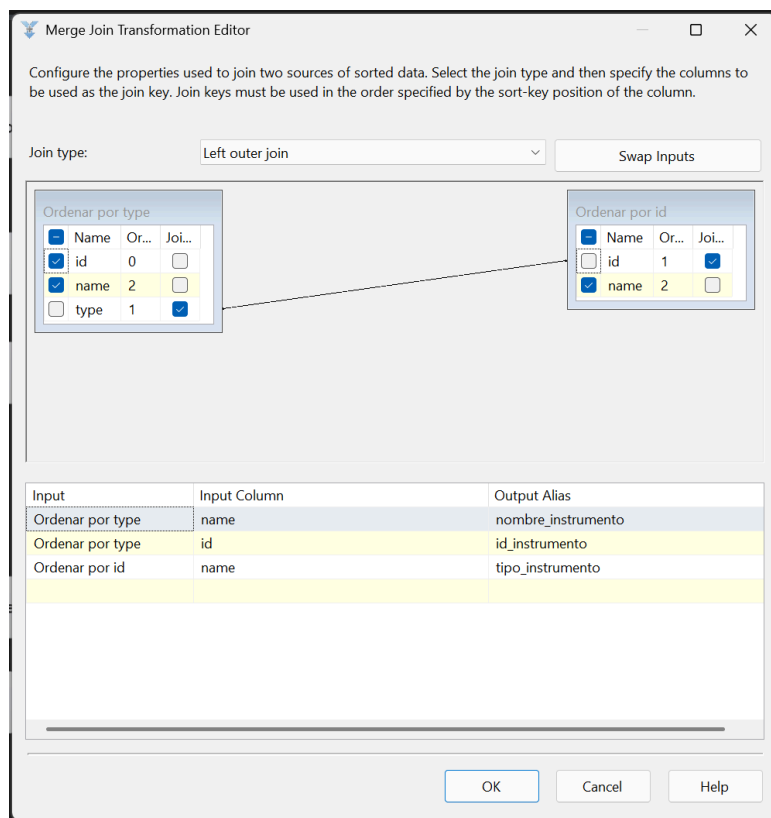
Configuración del Musicbrainz instrument.

De instrument, solo se necesitan las columnas id, name, y type. De instrument_type se necesitan las columnas id y name.

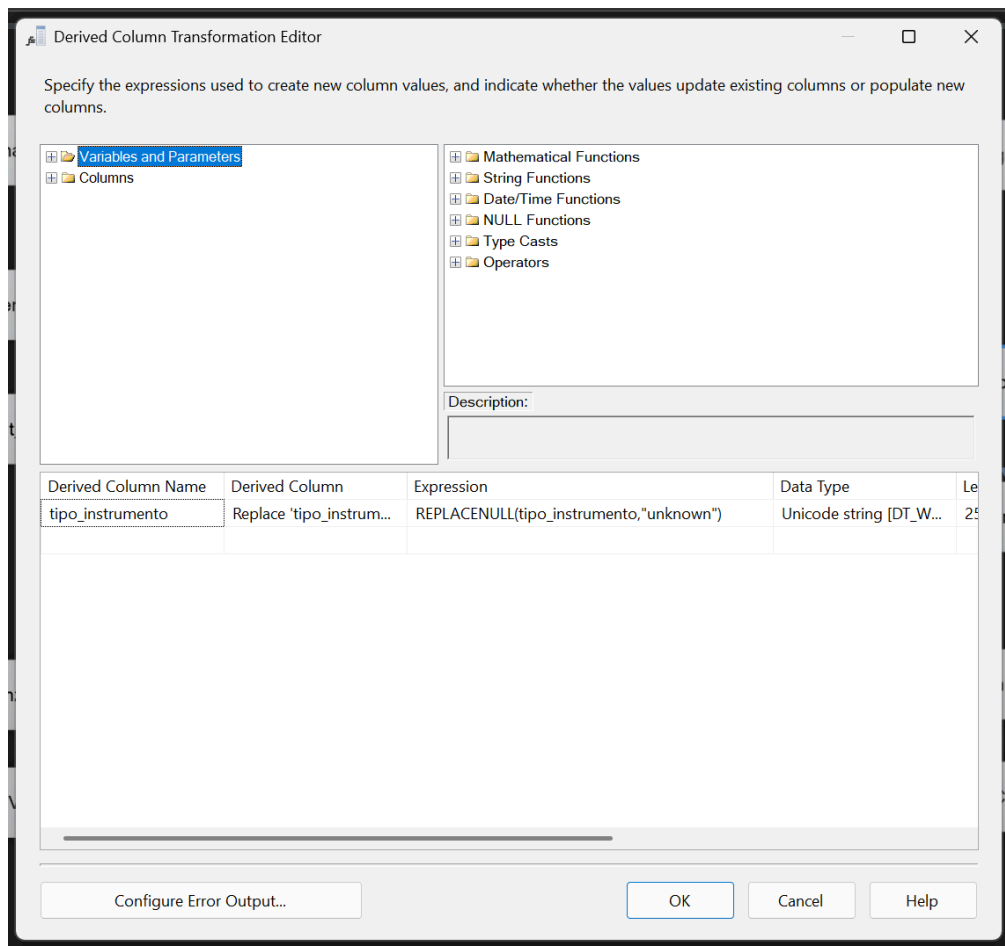
El componente Merge Join requiere que los inputs estén ordenados, por lo que se los ordena en los componentes Sort Transformation.



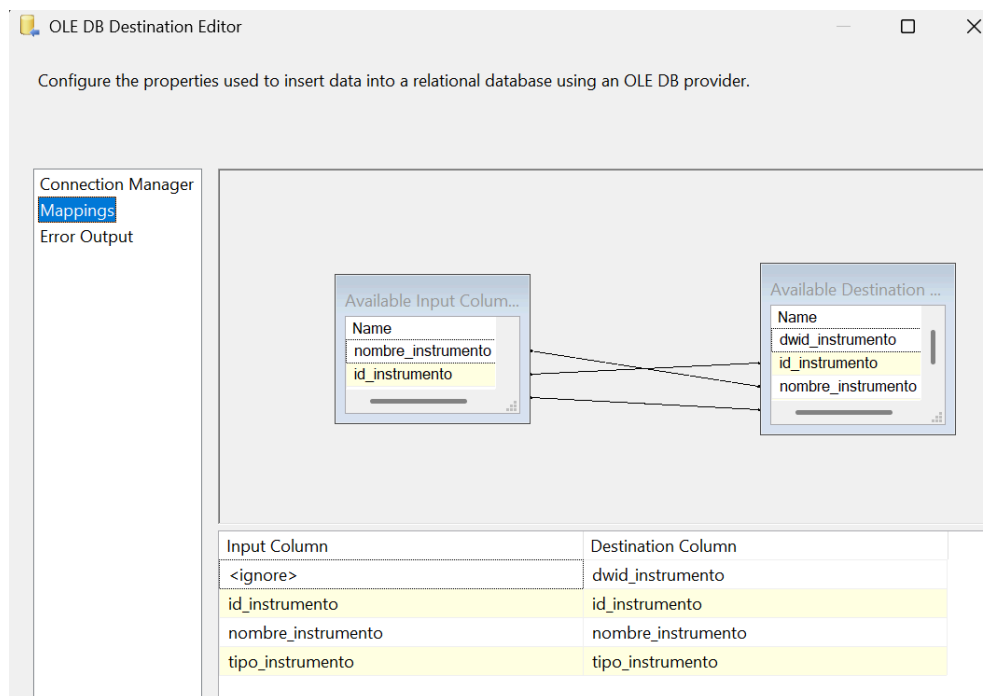
Configuración del componente Ordenar por type (sort order 1). En el componente Ordenar por id, el sort order 1 corresponde a la columna id.



Componente Merge Join.



Default tipo_instrumento se encarga de apuntar los NULLs al default "unknown".



Cargar DimInstrumento es más de lo mismo.

DimCategoria

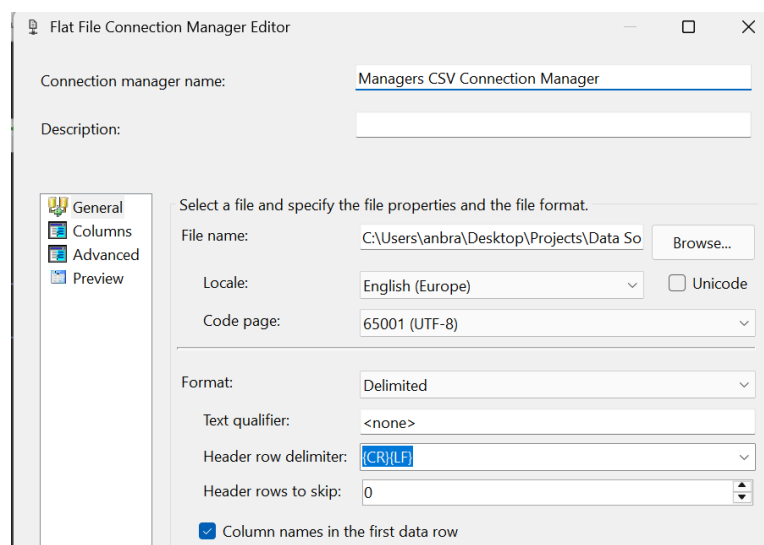


DimCategoria es simple. Solo interesan las columnas id y name de la tabla genre.

DimManager

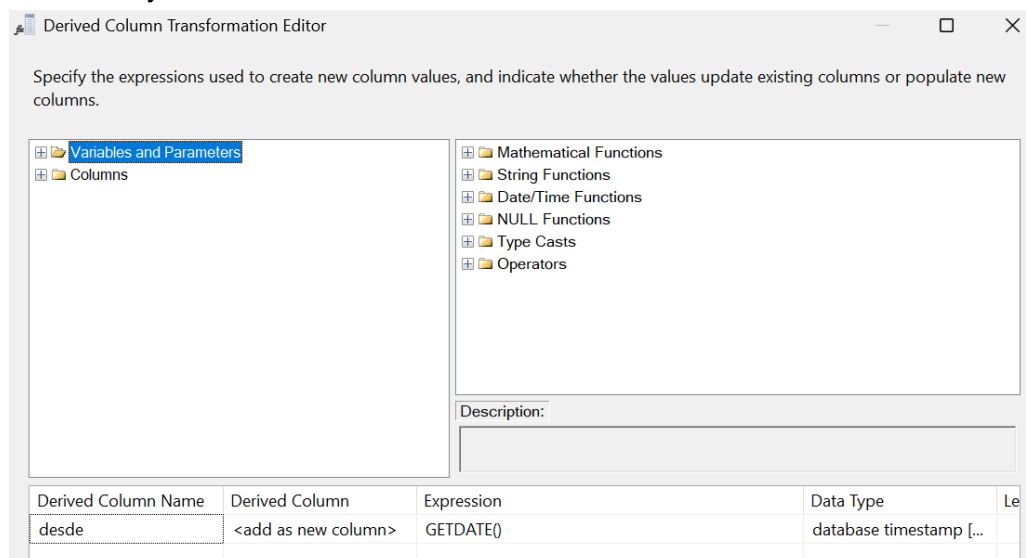


ETL de DimManager.



Conexión al managers.csv. Es importante la Code page 65001 UTF-8.

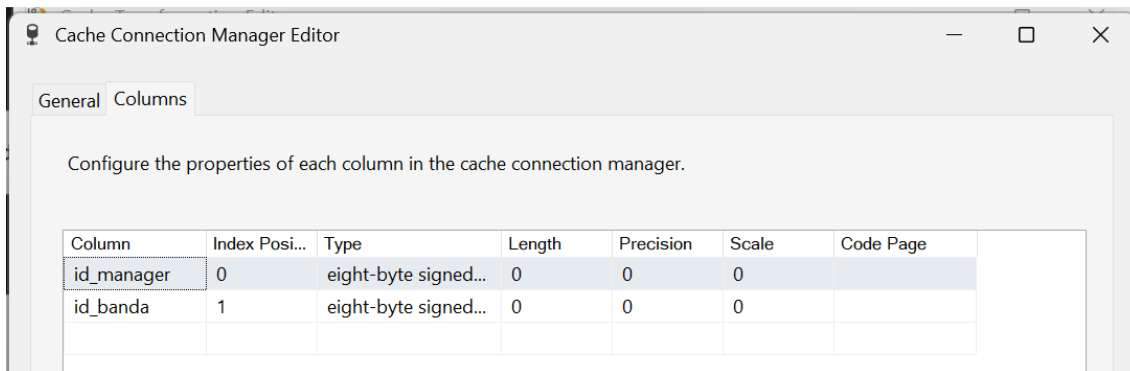
El managers CSV es mockeado por el script `mock.ipynb`. Actualmente ese archivo no tiene una columna desde y hasta.



Establecer Columna desde.

Hay que consultar con el profe si esto está bien, asignar el desde en el ETL. No tiene mucho sentido, lo lógico es que desde sea la fecha de carga en el sistema OLTP.

El CSV de managers tiene el id_banda asociado a cada manager. Necesitamos guardar esta información para luego consultarla al cargar las bandas. Las guardamos en un cache indexado por bandas. Esto requiere construir un nuevo connection manager a un cache con las siguientes columnas:

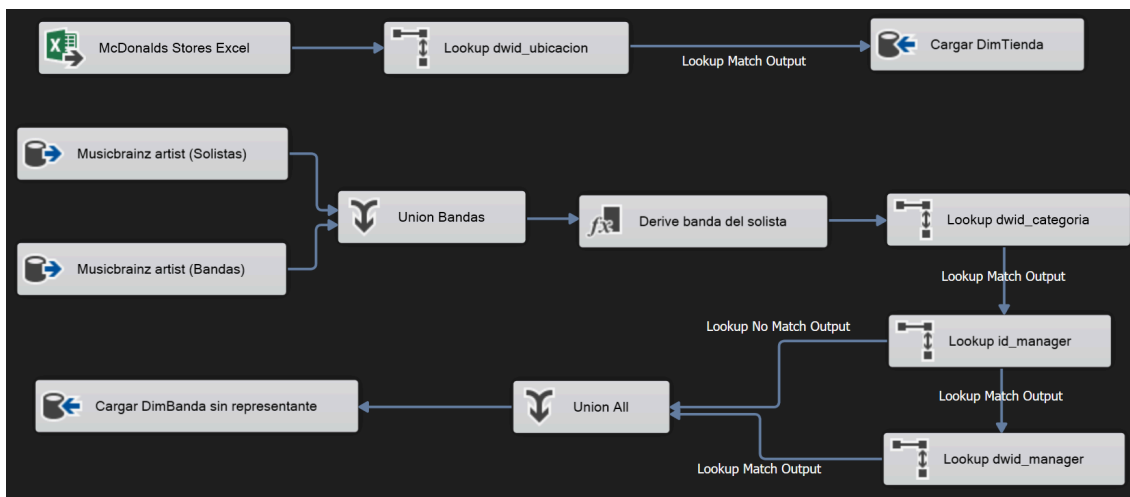


Column	Index Posi...	Type	Length	Precision	Scale	Code Page
id_manager	0	eight-byte signed...	0	0	0	
id_banda	1	eight-byte signed...	0	0	0	

En el ManagerBanda Cache Connection Manager, la columna id_manager es index.

Luego simplemente se lo carga en el componente Cargar DimManager.

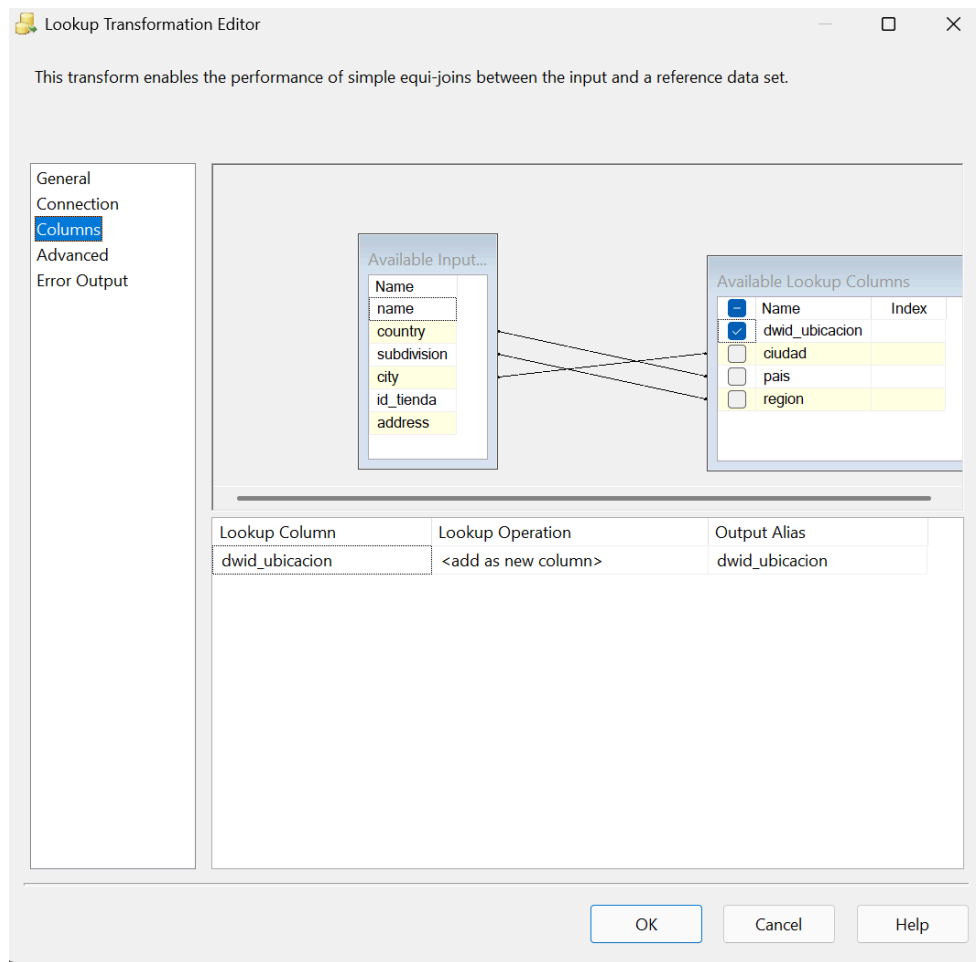
ETL DimTienda y DimBanda



Data Flow completo del ETL DimTienda y DimBanda.

DimTienda

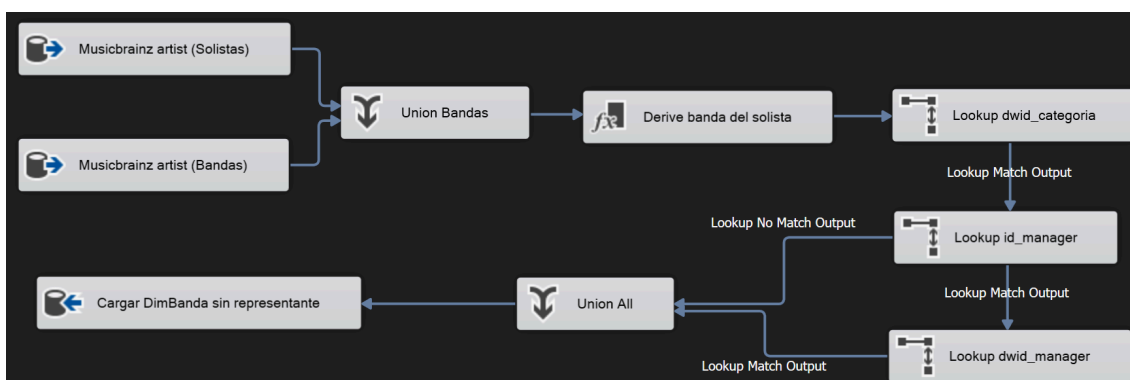
Se lee el Excel (Mc Donalds Stores), se hace un lookup contra el Data Warehouse discográfica para asociar las tiendas a las ubicaciones ya cargadas, y se cargan dichas tiendas. Este proceso es bastante simple, por lo que implicó ninguna configuración o ajuste adicional.



Lookup dwid_ubicacion para asociar cada tienda del Excel a la DimUbicación del DW.

DimBanda

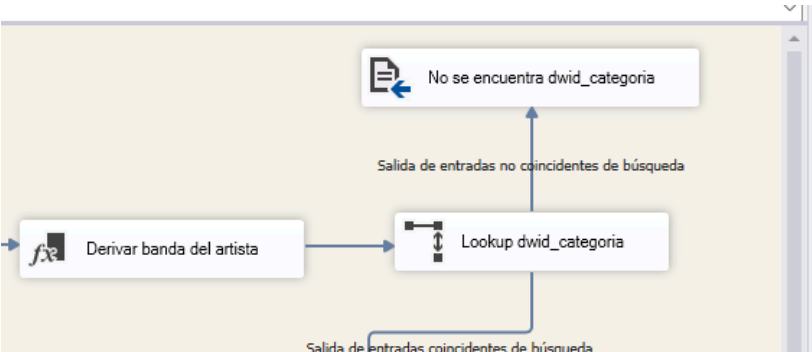
La dimensión Banda es mucho más compleja. En este flujo se cargan todas sus columnas excepto los representantes, que son una FK a los artistas (para el casos de aquellas bandas que tienen representantes, ya que también podrían tener manager), pero DimArtista todavía no se cargó.



ETL DimBanda.

Nota: algunos miembros del equipo no pudieron integrarse a musicbrainz desde SSIS, por lo que la fuente se reemplazó por un archivo CSV con los registros que la consulta SQL devuelve.

Se agrega una consideración: tratar el No Match Output de Lookup dwid_categoria para ver si hay alguna categoría que no matchea, redirigiendo los registros a un texto plano.



No Match Output de Lookup dwid_categoria.

En el archivo `QueriesMusicbrainz.sql` se encuentran la query *SOLISTAS* que va en Musicbrainz artist (Solistas), y la query *BANDAS* que va en Musicbrainz artist (Bandas).

Union All Transformation Editor

Configure the properties used to merge multiple inputs into one output by creating mappings between columns.

Output Column Name	Union All Input 1	Union All Input 2
nombre_banda	name	name
ended	ended	ended
categoria	categoria	categoria
pais_de_origen	pais_de_origen	pais_de_origen
cantidad_artistas	cantidad_artistas	<ignore>
id_banda	id	<ignore>
id_solista	<ignore>	id

El componente Union Bandas une ambos flujos.

Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

Variables and Parameters
Columns

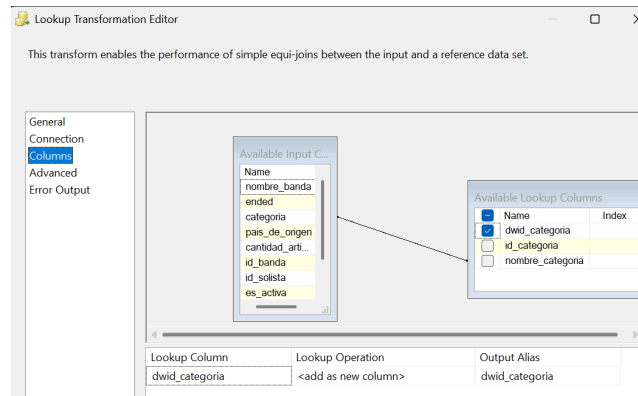
Mathematical Functions
String Functions
Date/Time Functions
NULL Functions
Type Casts
Operators

Description:

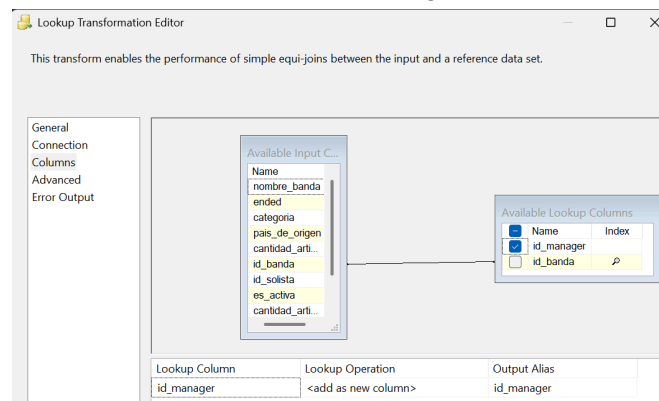
Derived Column Name	Derived Column	Expression	Data Type	Le
es_activa	<add as new column>	!ended	Boolean [DT_BOOL]	
cantidad_artistas_DT_I2	<add as new column>	(DT_I2)REPLACENULL(cantidad_artistas,1)	two-byte signed integ...	
categoria	Replace 'categoria'	REPLACENULL(categoria,"unknown")	Unicode string [DT_W...	25
pais_de_origen	Replace 'pais_de_orige...	REPLACENULL(pais_de_origen,"unknown")	Unicode string [DT_W...	25
id_banda	Replace 'id_banda'	REPLACENULL(id_banda,-id_solista)	eight-byte signed inte...	

Se deriva una banda para cada artista solista.

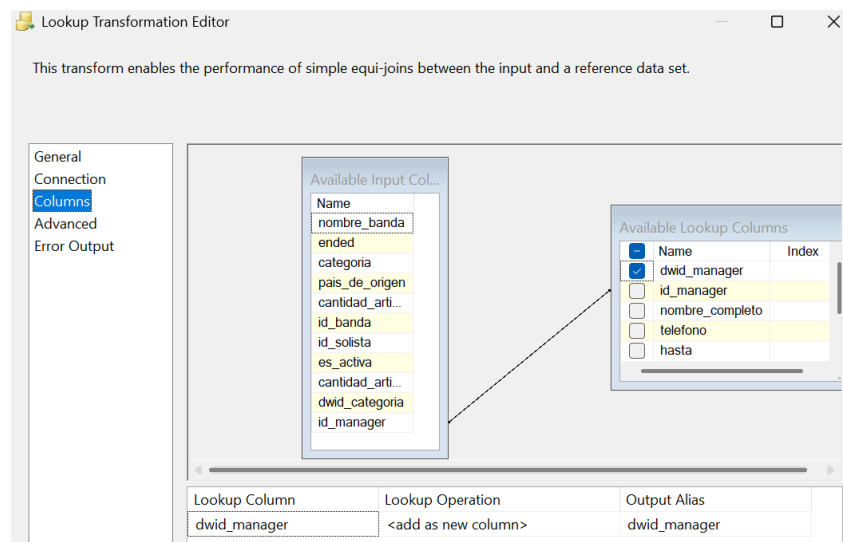
Se deriva banda del solista. A la banda "unipersonal" se le asigna el *id* del solista pero negativo. En el caso de consumir desde un CSV las expresiones de *REPLACENULL*(X, "unknown") la cambiamos por *(DT_WSTR,255)(ISNULL(X) || LEN(TRIM(X)) == 0 ? "unknown" : X)*



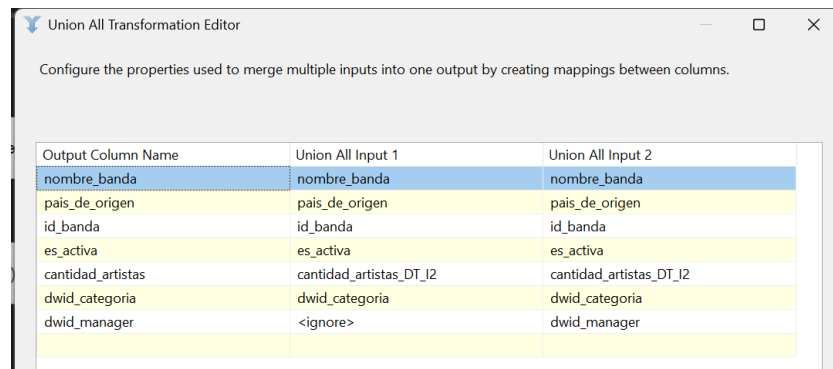
Lookup dwid_categoria.



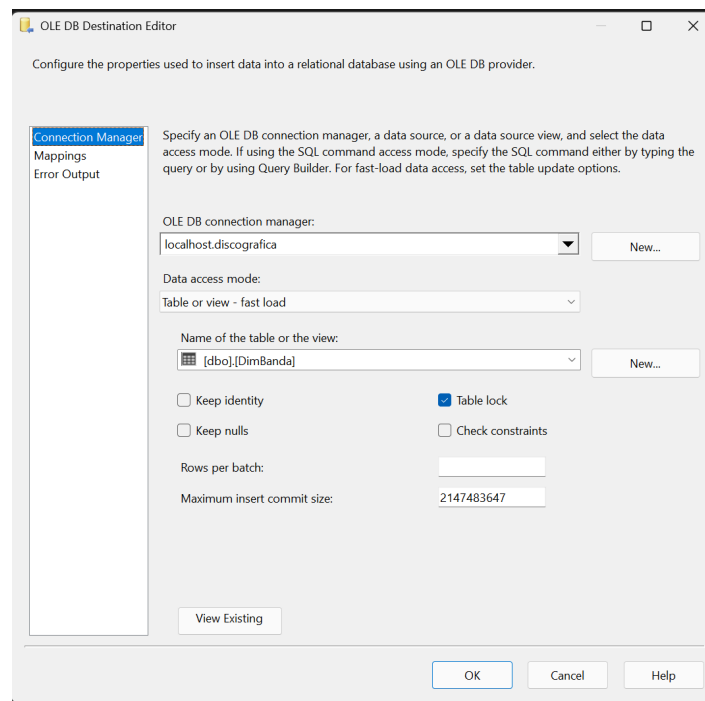
Lookup id_manager usa el cache creado al cargar managers en el flujo ETL Dimensiones Hoja.



Se realiza el lookup dwid_manager para las bandas que encontraron un id_manager.

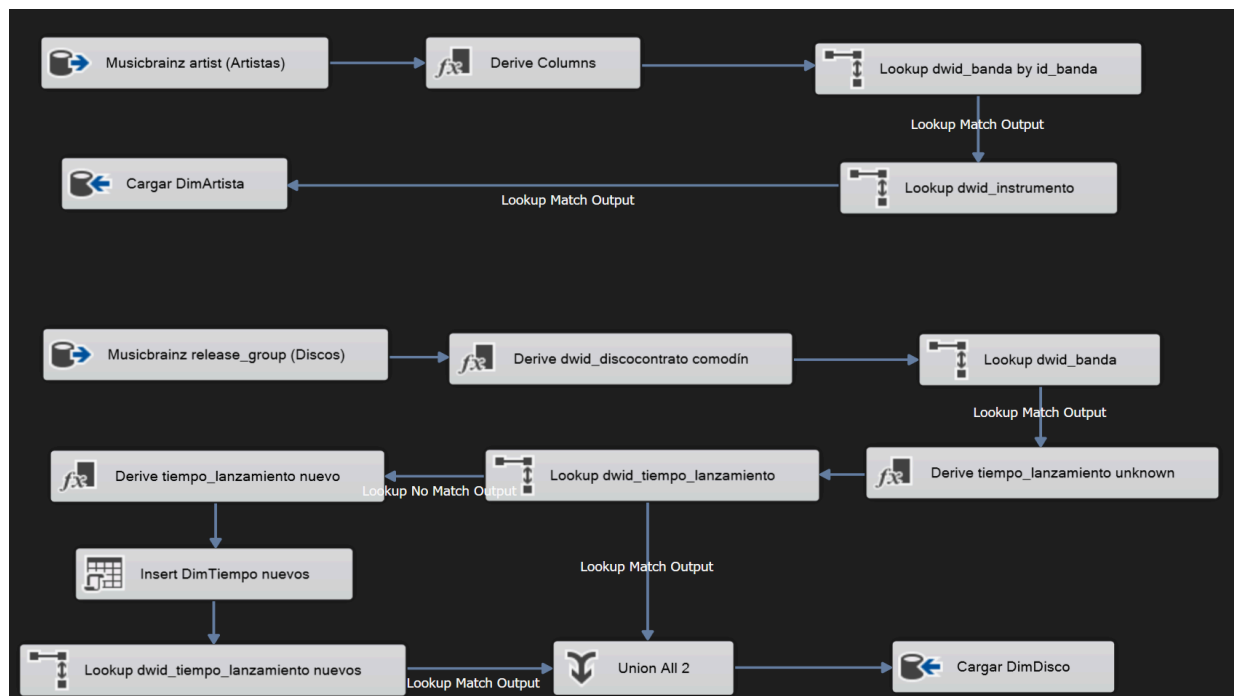


Union All deja pasar dwid_managers nulos porque es un campo opcional.



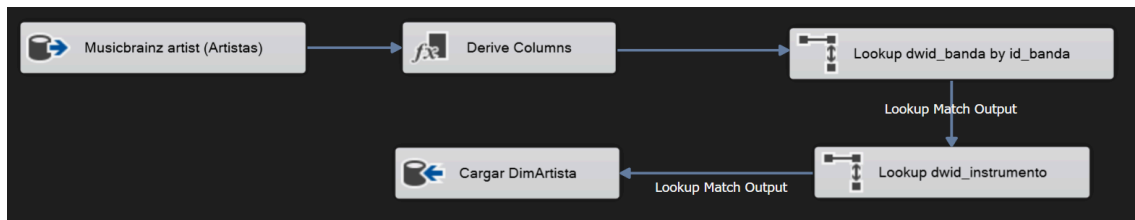
En el DW hay un check que valida que dwid_representante y dwid_manager no sean ambos nulos a la vez. Cargar DimBanda sin representante tiene deshabilitado el Check constraints porque por ahora se ignora la columna dwid_representante.

ETL DimArtista y DimDisco



ETL DimArtista y DimDisco.

DimArtista



DimArtista

En el componente ODBC Source va la query que se llama ARTISTAS PERSONA (solistas y miembros de bandas) del archivo `QueriesMusicbrainz.sql`.

Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

Variables and Parameters
Columns

Mathematical Functions
String Functions
Date/Time Functions
NULL Functions
Type Casts
Operators

Description:

Derived Column Name	Derived Column	Expression	Data Type	Le
es_activo	<add as new column>	(DT_BOOL)REPLACENULL(ended, FALSE)	Boolean [DT_BOOL]	
es_solista	<add as new column>	ISNULL(id_banda)	Boolean [DT_BOOL]	
desde	<add as new column>	GETDATE()	database timestamp [...]	
pais_de_origen	Replace 'pais_de_orige...	REPLACENULL(pais_de_origen,"unknown")	Unicode string [DT_W...	25
id_banda	Replace 'id_banda'	REPLACENULL(id_banda,-id)	eight-byte signed inte...	
instrumento	Replace 'instrumento'	REPLACENULL(instrumento,"unknown")	Unicode string [DT_W...	25
fecha_nacimiento	Replace 'fecha_nacimi...	REPLACENULL(fecha_nacimiento,"0001-01-01")	database date [DT_DB...	

Derive columns de un artista.

Lookup Transformation Editor

This transform enables the performance of simple equi-joins between the input and a reference data set.

General
Connection
Columns
Advanced
Error Output

Available Input Col...

Name
id
ended
nombre_completo
pais_de_origen
id_banda
instrumento
nombre_artistico
fecha_nacimiento
es_activo
es_solista

Available Lookup Columns

Name	Index
☒ dwid_banda	
☐ id_banda	
☐ nombre_banda	
☐ cantidad_arti...	
☐ pais_de_origen	
☐ es_activa	
☐ dwid_categoria	
☐ dwid_represe...	

Lookup Column	Lookup Operation	Output Alias
dwid_banda	<add as new column>	dwid_banda

Lookup dwid_banda by id_banda

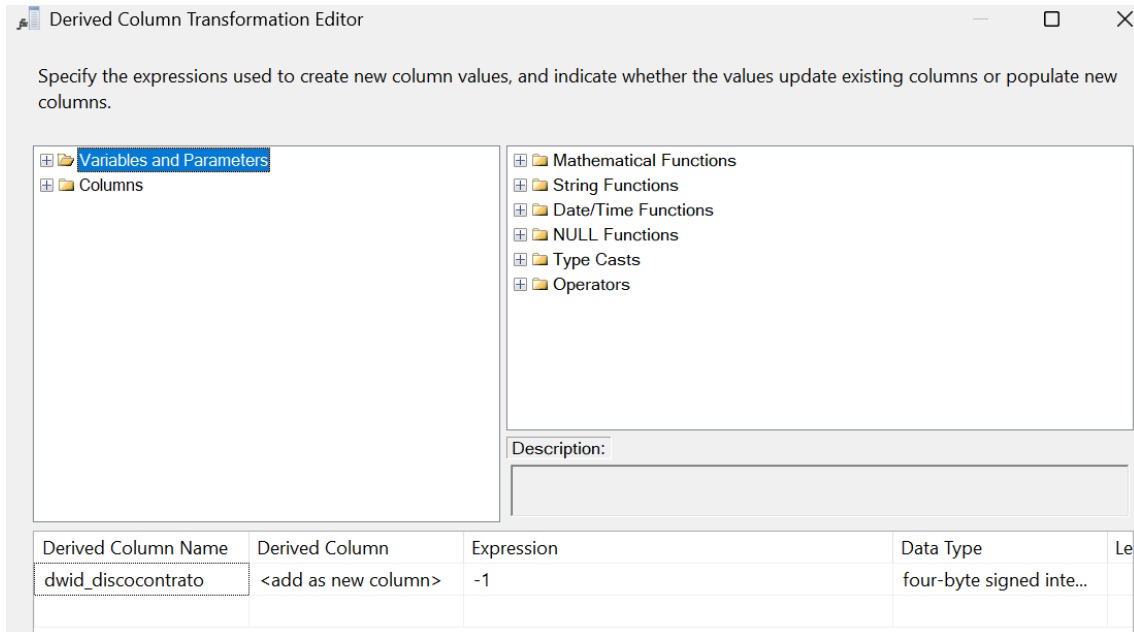
Cargar DimArtista es como la mayoría de OLE DB Destinations. Se mapean automáticamente todas las columnas.

DimDisco

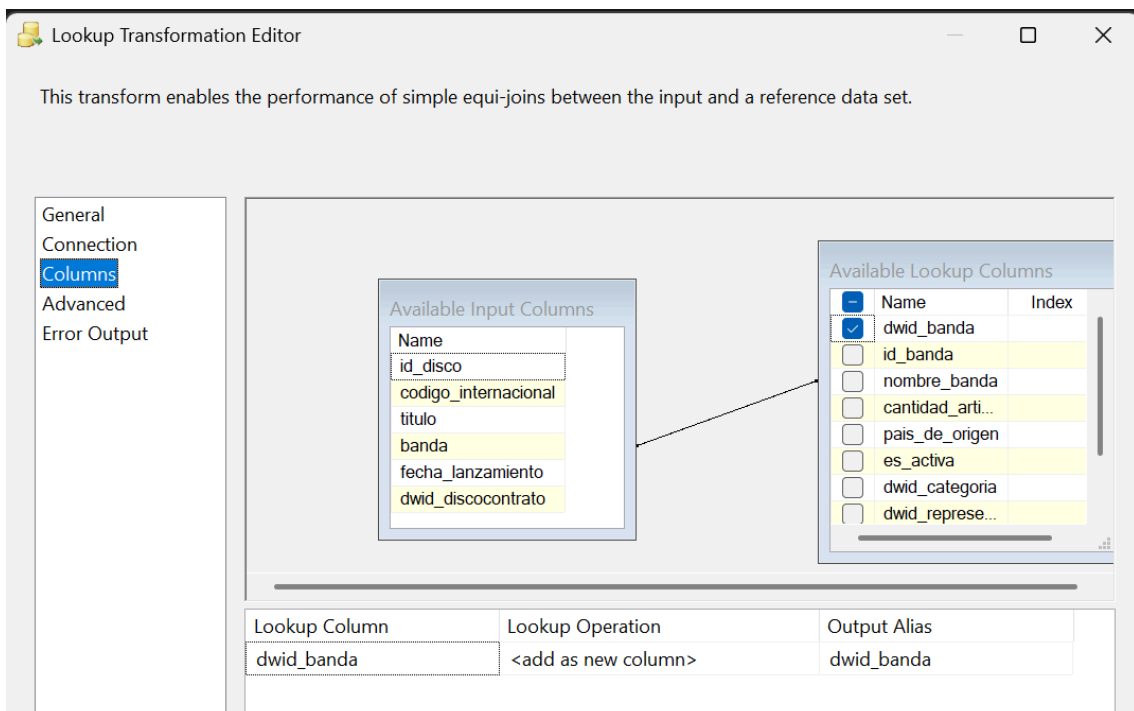
La carga de la dimensión Discos es simple pero algunas fechas de lanzamiento eran inusuales encontrándose, fuera del rango 1888-2045 que está cargado en DimTiempo. Decidimos considerar como erróneos los tiempos inferiores a 01-01-1800.

En Musicbrainz release_group (Discos) se usa la query DISCOS que se encuentra en el archivo

■ `QueriesMusicbrainz.sql` .

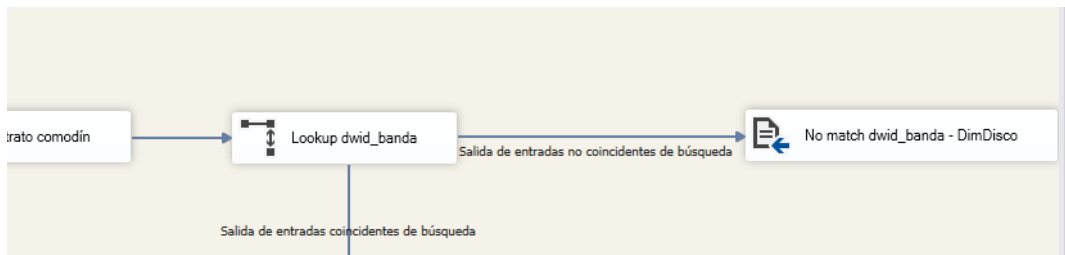


Derive dwid_discocontrato comodín



Lookup dwid_banda.

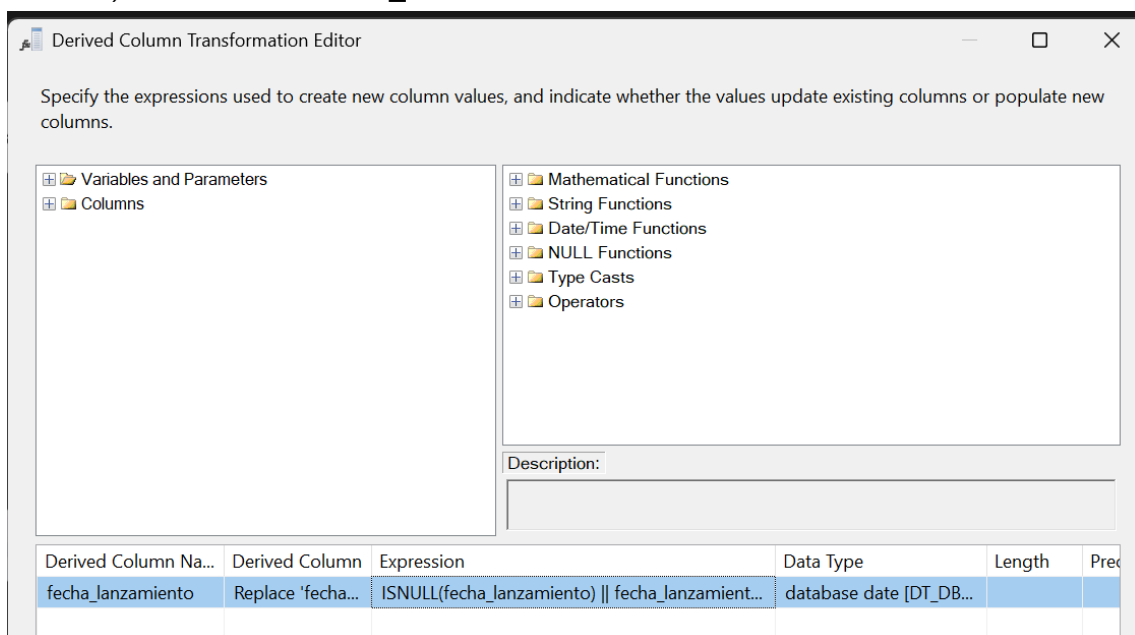
Al esquema anterior se le agrega luego del lookup dw dwid_banda una redirección de las salidas no coincidentes a un texto plano para luego evaluar la razón por la cual no coincidieron.



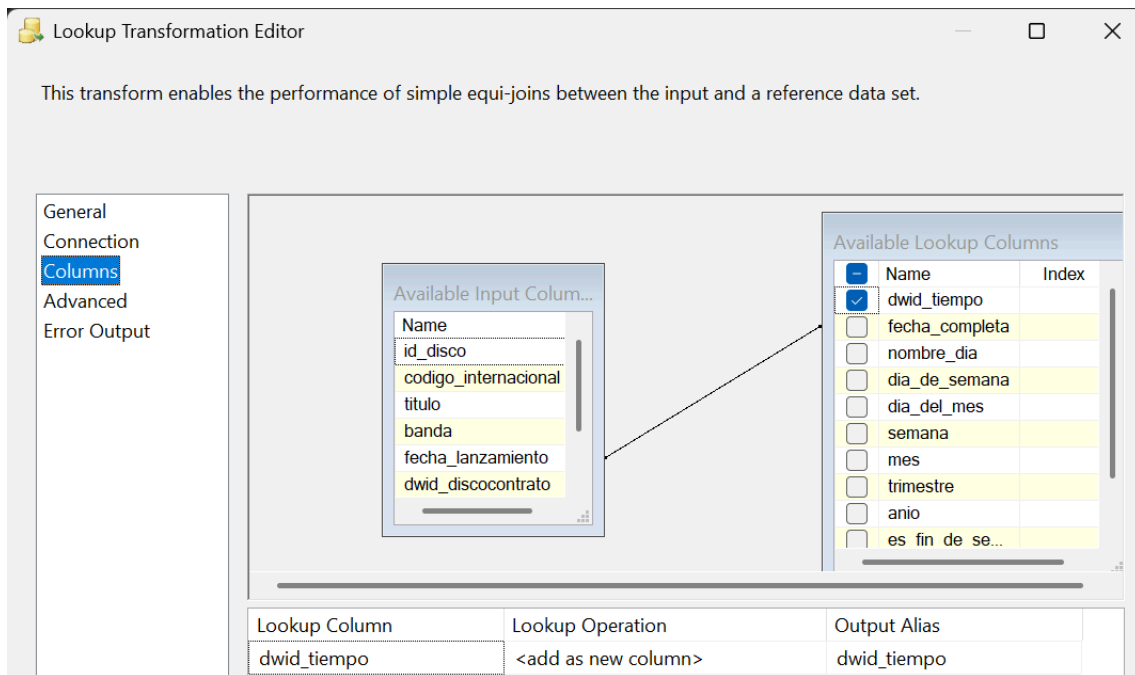
Loguear los registros que no hicieron match en el Lookup dwid_banda.

Se usa la siguiente expresión para limpiar los datos de fecha_lanzamiento, ya que con esto poblamos aquellos registros con valores nulos o con fechas consideradas erróneas. La expresión para hacer esto es la siguiente:

ISNULL(fecha_lanzamiento) || fecha_lanzamiento < (DT_DBDATE)"1800-01-01" ? (DT_DBDATE)"0001-01-01" : fecha_lanzamiento

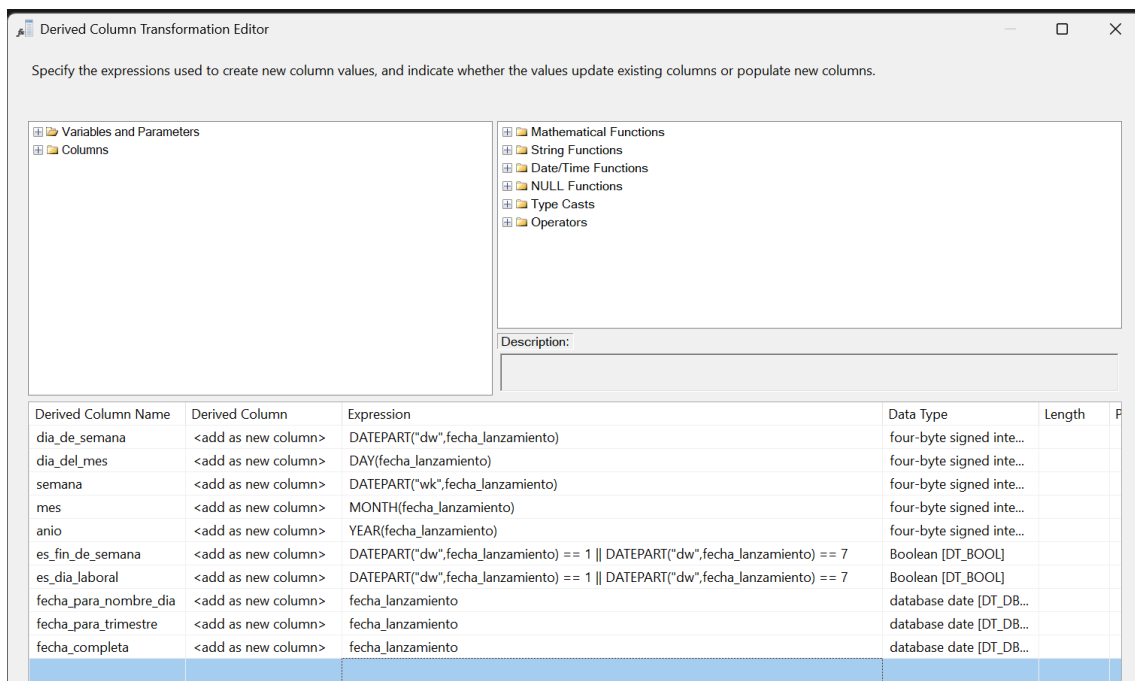


Derive tiempo_lanzamiento unknown.



Lookup dwid_tiempo_lanzamiento.

Los discos que no tienen una fecha de lanzamiento válida apuntan al DimTiempo comodín "0001-01-01" para indicar que es una fecha desconocida. No descartamos ninguna fila por no tener una fecha de lanzamiento válida.



Derive tiempo_lanzamiento nuevo.

En Insert DimTiempo nuevos > Component Properties > SqlCommand va el comando INSERTAR DIMTIEMPOS NUEVOS que se encuentra en QueriesDiscografica.sql .

Todas estas columnas derivadas que se crearon para insertar los nuevos tiempos luego se mapearon con los parámetros de la consulta SQL.

Advanced Editor for Insert DimTiempo nuevos

The advanced editor provides access to the low-level properties of data flow components. Additionally, the advanced editor can be used to configure components that do not have a custom user interface.

Connection Managers Component Properties **Column Mappings** Input and Output Properties

Available Input Columns

Name
id_disco
codigo_internacional
titulo
banda
fecha_lanzamiento
dwid_discocontrato
dwid_banda
dia_de_semana
dia_del_mes
semana
mes
anio
es_fin_de_semana
es_dia_laboral
fecha_para_nombre...
fecha_para_trimestre...
fecha_completa

Available De...

Name
Param_0
Param_1
Param_2
Param_3
Param_4
Param_5
Param_6
Param_7
Param_8
Param_9
Param...

Input Column	Destination Column
fecha_lanzamiento	Param_0
fecha_completa	Param_1
dia_de_semana	Param_2
fecha_para_nombre_dia	Param_3
dia_del_mes	Param_4
semana	Param_5
mes	Param_6
fecha_para_trimestre	Param_7
anio	Param_8
es_fin_de_semana	Param_9
es_dia_laboral	Param_10

Column Mappings para Insert DimTiempo nuevos. Cada Param_X es un '?' en el SQL.

General
Connection
Columns
Advanced
Error Output

Available Input Column...

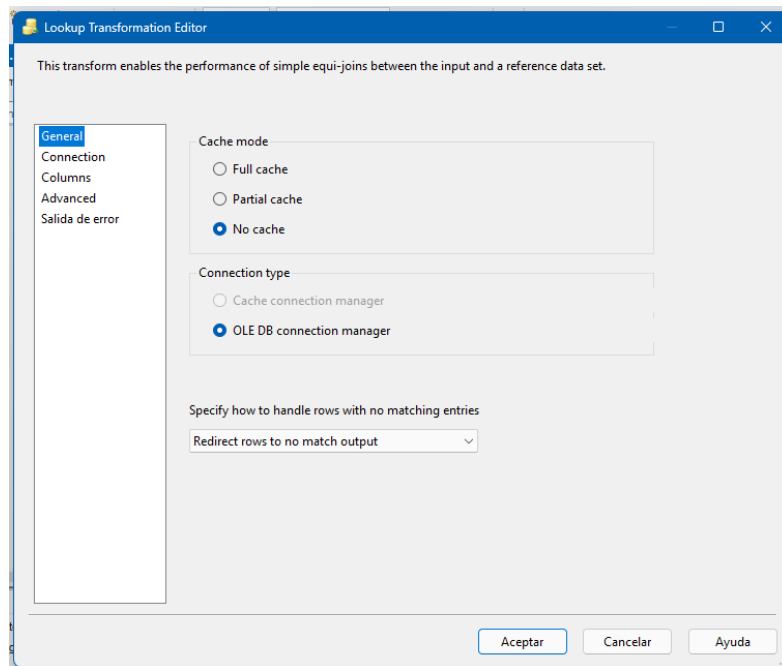
Name
id_disco
codigo_internacional
titulo
banda
fecha_lanzamiento
dwid_discocontrato
dwid_banda
dia_de_semana
dia_del_mes
semana

Available Lookup Columns

☐	Name	Index
☒	dwid_tiempo	
☐	fecha_completa	
☐	nombre_dia	
☐	dia_de_semana	
☐	dia_del_mes	
☐	semana	
☐	mes	
☐	trimestre	
☐	anio	
☐	es fin de se...	

Lookup Column	Lookup Operation	Output Alias
dwid_tiempo	<add as new column>	dwid_tiempo

Lookup dwid_tiempo by fecha_completa = fecha_lanzamiento.



Lookup en modo No cache.

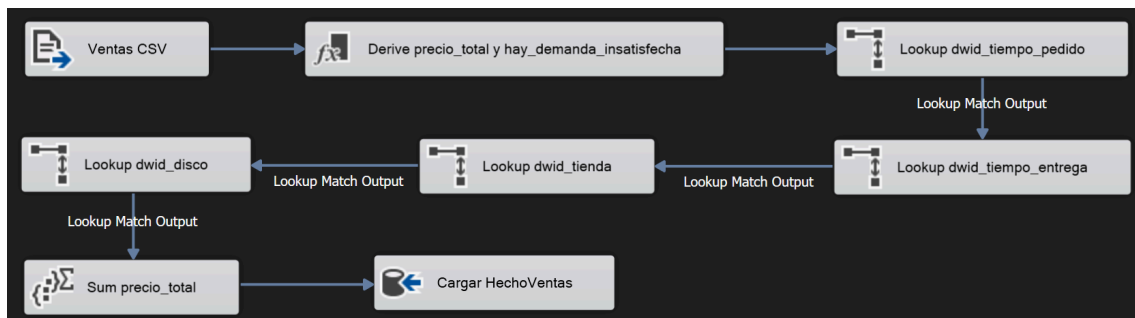
Lookup dwid_tiempo_lanzamiento nuevos. Es importante que este Lookup esté en modo *No Cache* para que se ejecute en el momento en que se llega al componente. Caso contrario, un *Full Cache* trae la info de la tabla a la que se hará el lookup al inicio de la ejecución completa, antes de llegar al componente. En ese caso no se contaría todavía con las nuevas fechas que se cargan en el componente anterior.

Luego se unen los Lookup Match Outputs en el Union All 2 y van directo al Cargar DimDisco.

Asignar representantes en DimBanda

Este paso es un Execute SQL Task con la query `ASIGNAR REPRESENTANTES A BANDAS (ASEGURAR QUE TODOS LOS REGISTROS TENGAN REPRESENTANTE SI NO TIENEN MANAGER)` que se puede encontrar en el archivo `QueriesDiscografica.sql`.

Las bandas creadas para solistas y registradas sin managers todavía no tienen su representante. Este paso se encarga de asignar el primer miembro de la banda encontrado en el DW como representante. Entendemos que esto es arbitrario y no como funcionaría en un negocio real, pero como no tenemos una fuente de datos con representantes, lo solucionamos de esta manera.

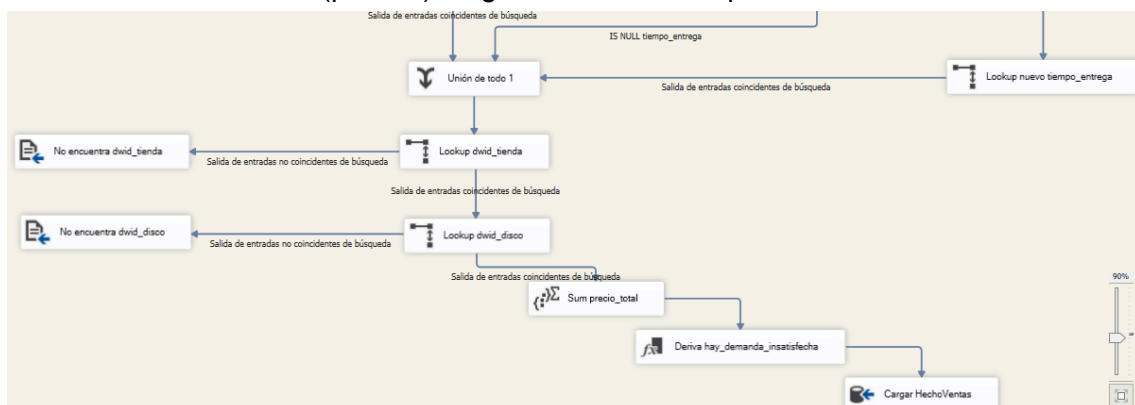


ETL HechoVentas básico.

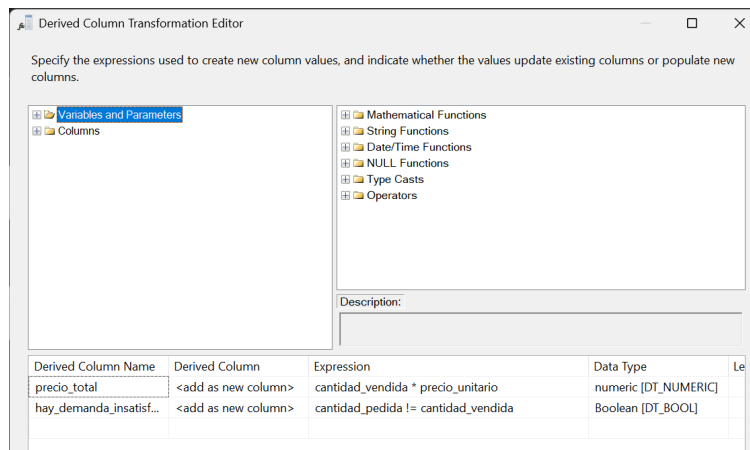
Al ETL HechoVentas básico le agregamos consideraciones sobre fechas no encontradas. Este enfoque consiste en cargar en DimTiempo las fechas no encontradas. Otro enfoque sería considerar inválidas a esas fechas no encontradas, ya que serían previas a 1888 o posteriores a 2045.



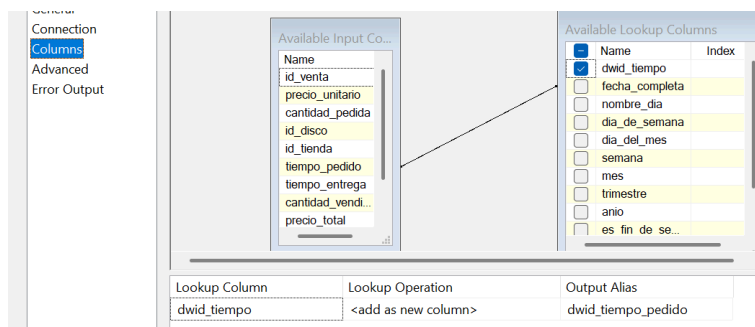
ETL HechoVentas (parte 1) cargando en DimTiempo fechas no encontradas.



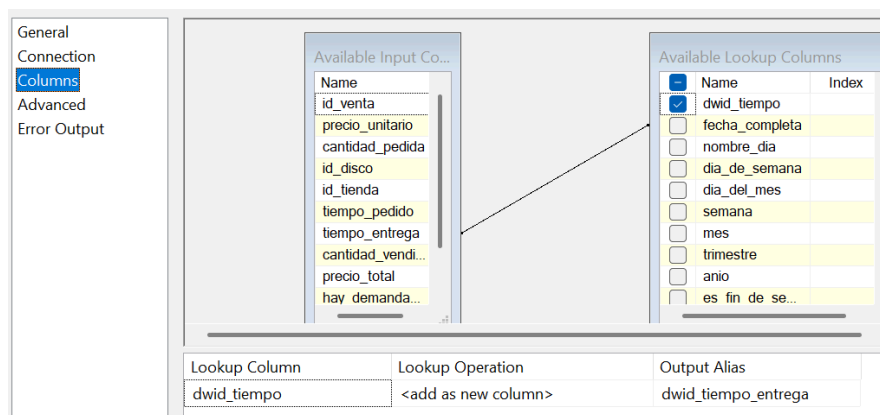
ETL HechoVentas (parte 2) con logging de los registros sin dwid_banda o dwid_disco.



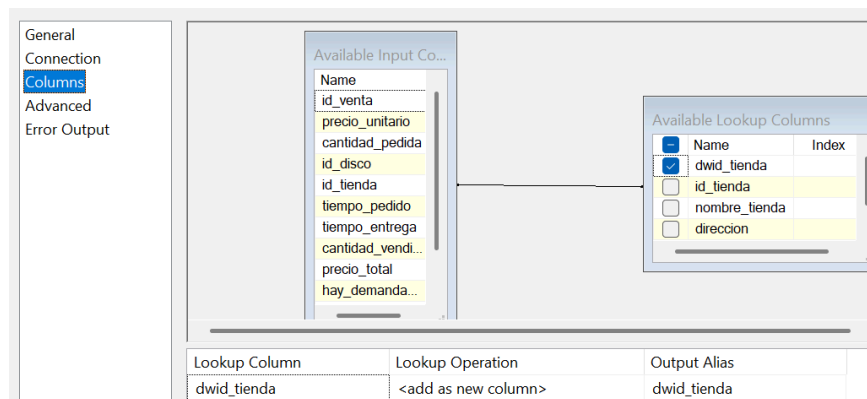
Derive precio_total y hay_demanda_insatisfecha.



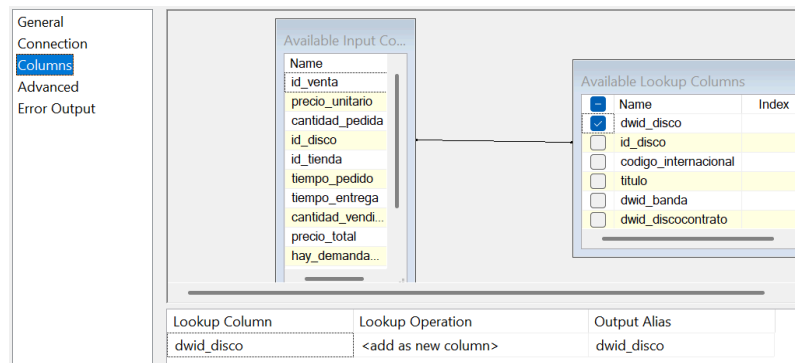
Lookup dwid_tiempo_pedido.



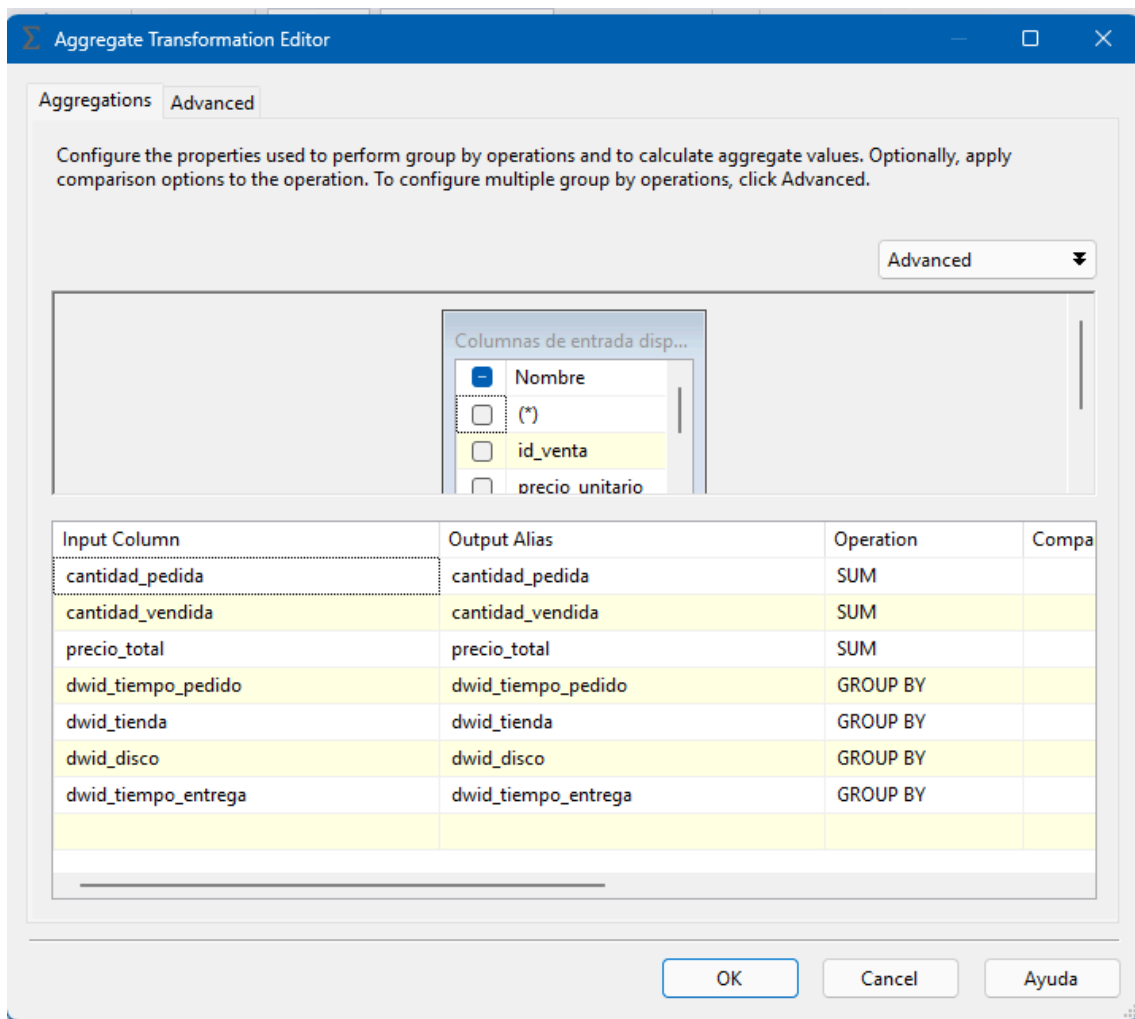
Lookup dwid_tiempo_entrega.



Lookup dwid_tienda.



Lookup dwid_disco.



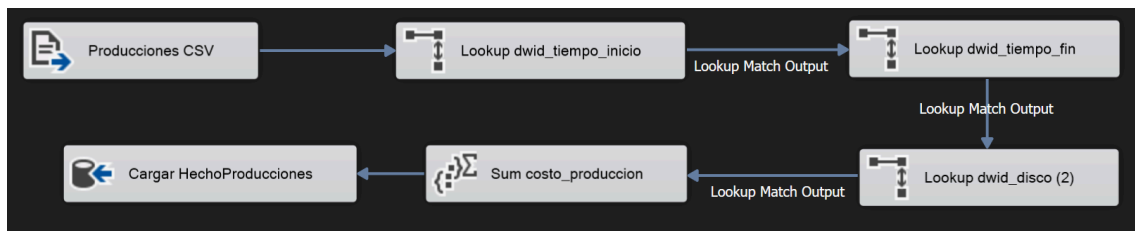
Sum precio_total.

En el aggregate se suman los precios y las cantidades, para luego, en el siguiente paso, poder calcular si hay demanda insatisfecha. Finalmente en Cargar HechoVentas se cargan los registros al DW.

HechoProducciones

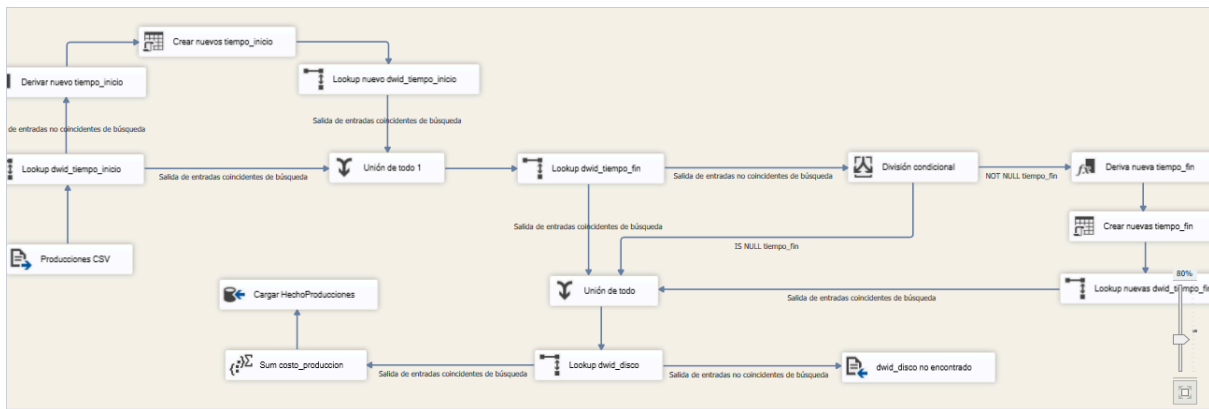
La carga del hecho Producciones no difiere mucho de lo visto en Ventas, salvo por las dimensiones con las cuales se relaciona. En primer lugar se tiene en cuenta de crear los tiempos

que no existan para las dimensiones de tiempo_inicio y tiempo_fin y de informar en el caso de las salidas no coincidentes del lookup para la dimensión de disco.

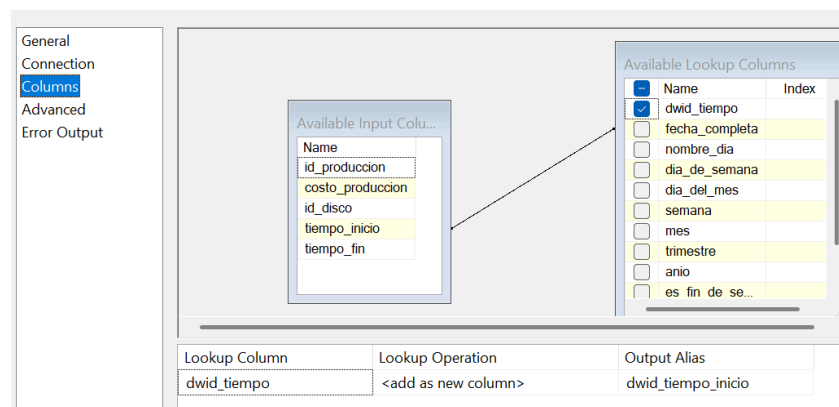


ETL HechoProducciones básico.

Al esquema básico le agregamos consideraciones para fechas no encontradas. Este enfoque consiste en cargar en DimTiempo las fechas no encontradas. Otro enfoque sería considerar inválidas a esas fechas no encontradas, ya que serían previas a 1888 o posteriores a 2045.



ETL HechoProducciones cargando en DimTiempo las fechas no encontradas.



Lookup dwid_tiempo_inicio.

General
Connection
Columns
Advanced
Error Output

Available Input Col...

Name
id_produccion
costo_produccion
id_disco
tiempo_inicio
tiempo_fin

Available Lookup Columns

Name	Index
☒ dwid_tiempo	
☐ fecha_completa	
☐ nombre_dia	
☐ dia_de_semana	
☐ dia_del_mes	
☐ semana	
☐ mes	
☐ trimestre	
☐ anio	
☐ es fin de se...	

Lookup Column	Lookup Operation	Output Alias
dwid_tiempo	<add as new column>	dwid_tiempo_fin

Lookup dwid_tiempo_fin.

General
Connection
Columns
Advanced
Error Output

Available Input Col...

Name
id_produccion
costo_produccion
id_disco
tiempo_inicio
tiempo_fin
dwid_tiempo_ini...

Available Lookup Columns

Name	Index
☒ dwid_disco	
☐ id_disco	
☐ codigo_internacional	
☐ titulo	
☐ dwid_banda	
☐ dwid_discocontrato	

Lookup Column	Lookup Operation	Output Alias
dwid_disco	<add as new column>	dwid_disco

Lookup dwid_disco.

Aggregations **Advanced**

Configure the properties used to perform group by operations and to calculate aggregate values. Optionally, apply comparison options to the operation. To configure multiple group by operations, click Advanced.

Advanced ▼

Available Input Columns

Name
☒ Name
☐ (*)
☒ id_produccion
☒ costo_produccion
☐ id_disco
☐ tiempo_inicio
☐ tiempo_fin
☒ dwid_tiempo_inicio

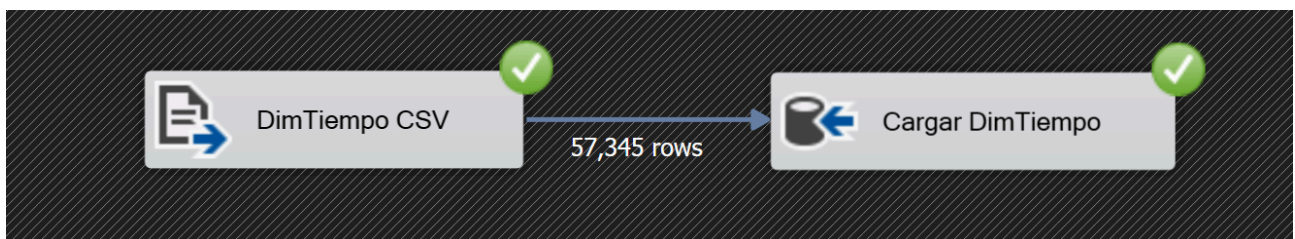
Input Column	Output Alias	Operation	Compa
dwid_tiempo_fin	dwid_tiempo_fin	Group by	
dwid_disco	dwid_disco	Group by	
dwid_tiempo_inicio	dwid_tiempo_inicio	Group by	
costo_produccion	costo_produccion	Sum	
id_produccion	id_produccion	Group by	

Sum costo_produccion.

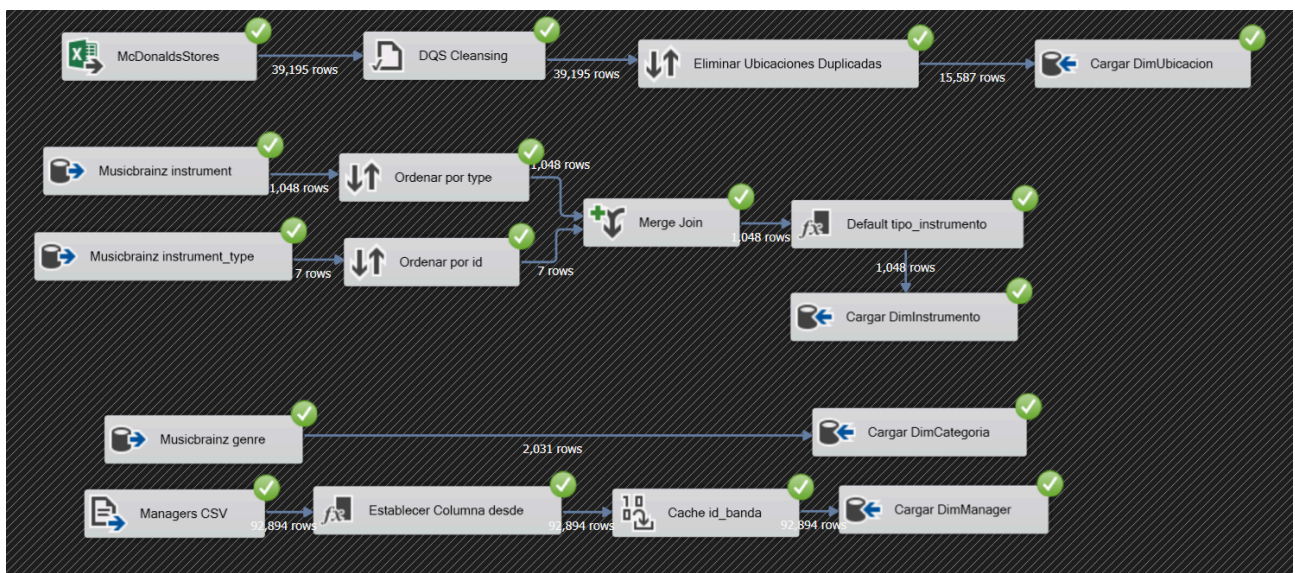
Ejecución de la Carga Inicial



Control Flow de la Carga Inicial.



ETL DimTiempo.

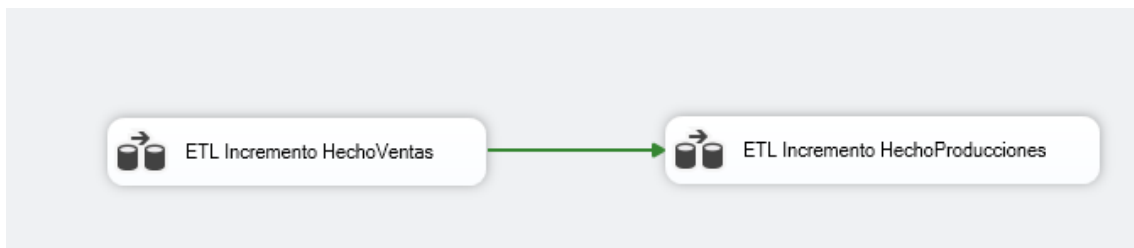


ETL Dimensiones Hoja.



ETL Hechos.

Carga Incremental



Esquema de la carga incremental.

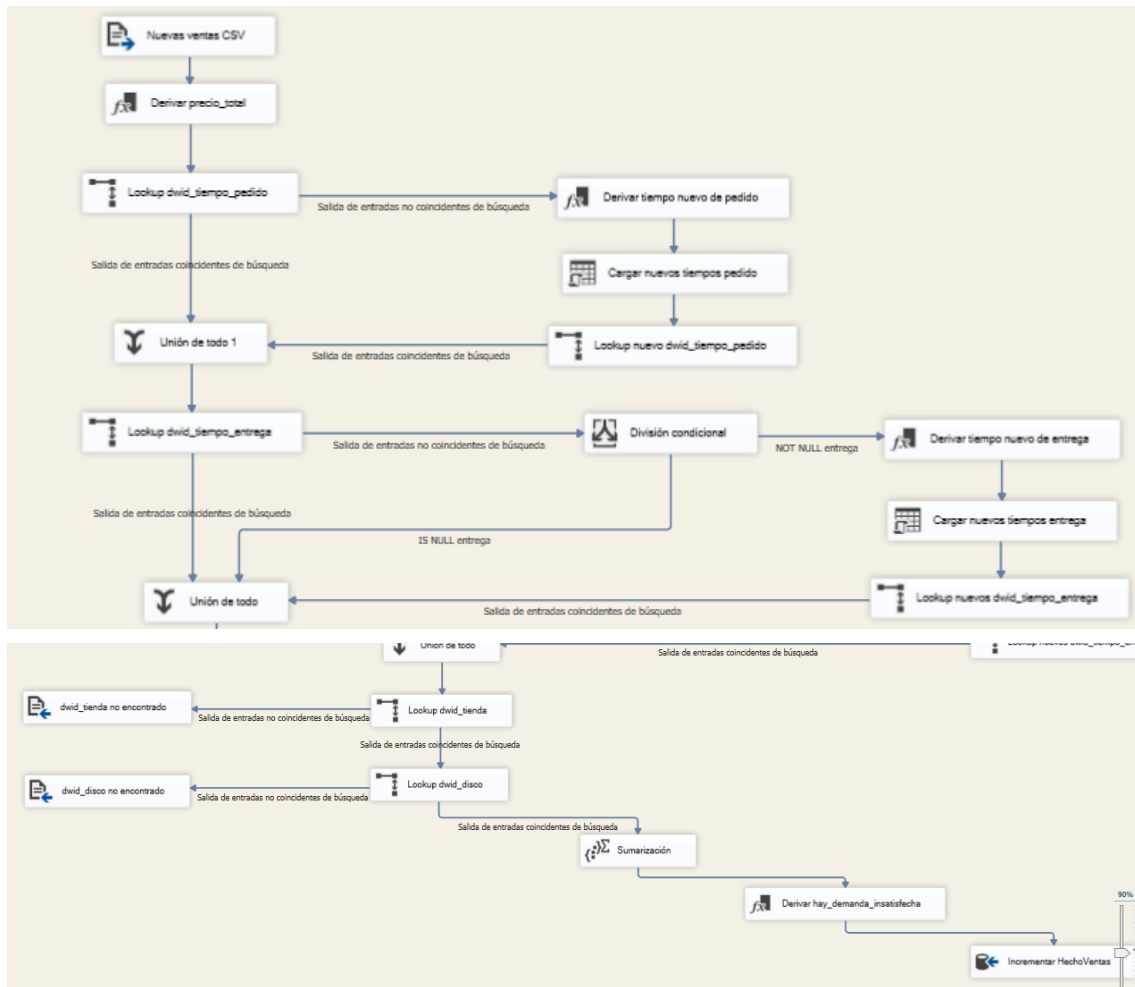
Una vez finalizada la carga inicial del Data Warehouse, es fundamental incorporar un mecanismo que permita actualizar la información de manera eficiente a medida que nuevos datos estén disponibles en los sistemas de origen. Es aquí donde entra en juego la carga incremental, cuyo objetivo principal es detectar y procesar únicamente los cambios o adiciones recientes, evitando así reprocesar grandes volúmenes de datos innecesariamente.

Esto permite mantener el DW actualizado, favoreciendo la toma de decisiones con información actual. Para ello, se diseñaron flujos específicos que identifican nuevos registros en las fuentes de datos y cargan los datos.

En los casos donde surgen valores faltantes (por ejemplo, nuevas fechas no presentes en la dimensión temporal), el flujo incluye ramificaciones que permiten generar estos registros “al vuelo” y reintentar la carga. Además, se prevé la documentación de los errores mediante archivos de salida que registran los datos no cargados por inconsistencias, lo que facilita su análisis posterior.

A continuación se muestra la implementación para las dos tablas de hechos principales del sistema: HechoVentas y HechoProducciones.

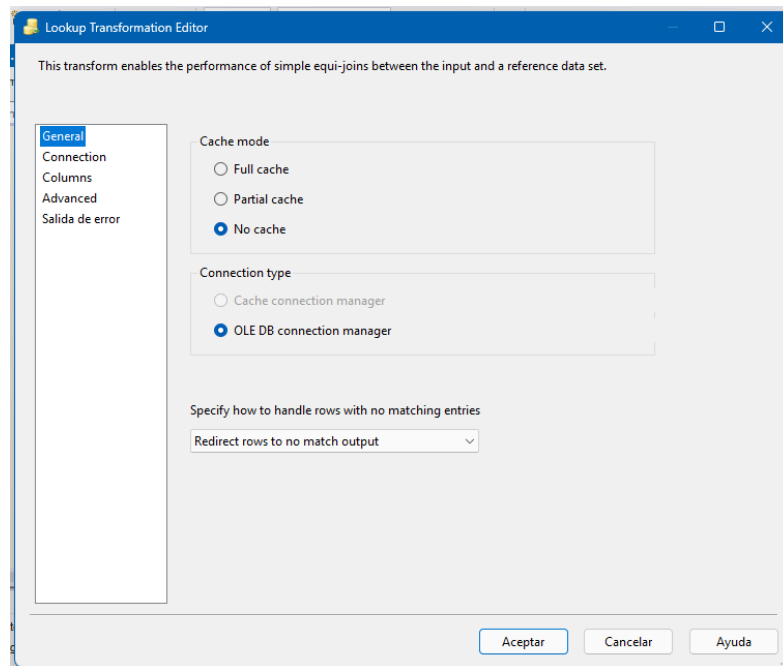
ETL Incremento HechoVentas



ETL Incremental HechoVentas.

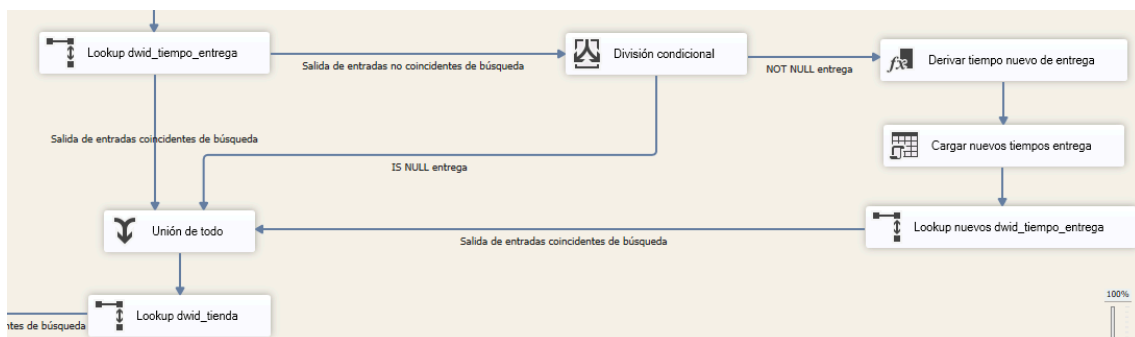
Este flujo es bastante similar a lo visto en la carga inicial de HechoVentas.

Una parte clave de este flujo consiste en la gestión de las fechas involucradas. En particular, se realiza un lookup sobre la dimensión temporal (DimTiempo) para obtener el identificador correspondiente al campo tiempo_pedido. Cuando se detectan fechas nuevas, o sea aquellas no cargadas en la dimensión, se deriva automáticamente un flujo para generar esos registros faltantes y cargarlos a DimTiempo. Una vez incorporados, se retoma el proceso con un nuevo lookup para asociar correctamente el identificador (dwid_tiempo_pedido) a cada registro de venta.

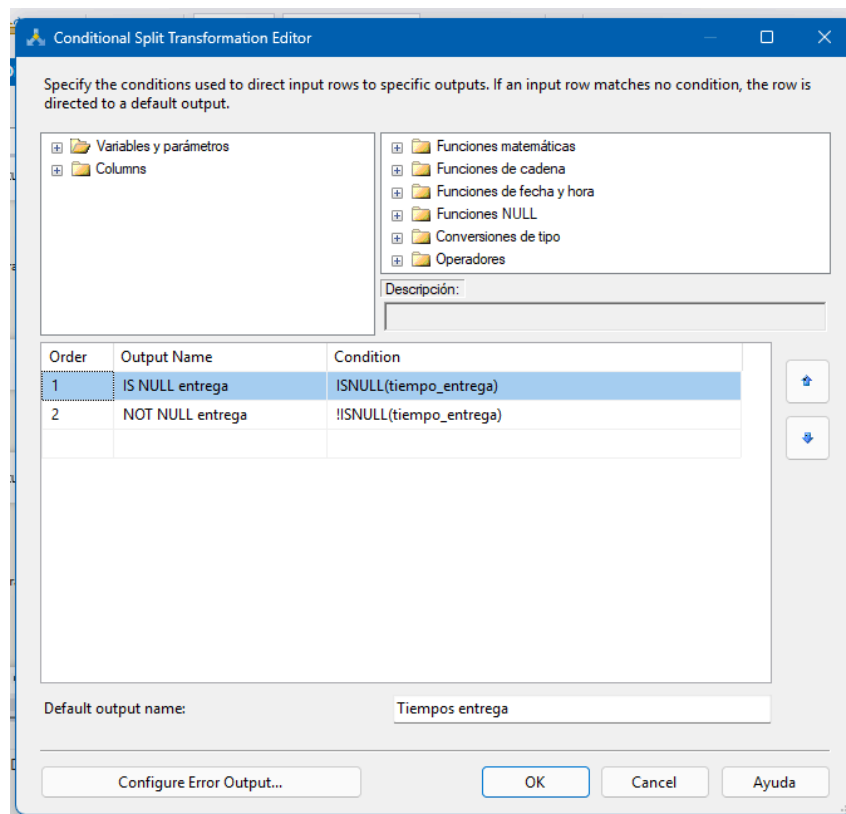


Lookup en modo No cache.

Un principio importante del que nos dimos cuenta en estos lookups es que en todos los ellos, en los que se hagan después de alguna carga de datos, es importante que se ejecuten en modo "No Cache", ya que un Full Cache consultaría la tabla solo al inicio del paquete, impidiendo detectar los nuevos registros generados dinámicamente durante el mismo flujo.



Una lógica similar se aplica al campo tiempo_entrega, pero considerando una particularidad adicional: este campo puede ser NULL, lo cual representa un caso válido. Para contemplar esta situación, se incorpora un flujo condicional que divide los registros en dos rutas: aquellos con fecha de entrega, para los cuales se realiza nuevamente el proceso de carga y asociación del tiempo si es necesario, y aquellos sin fecha de entrega, que siguen adelante sin necesidad de lookup.

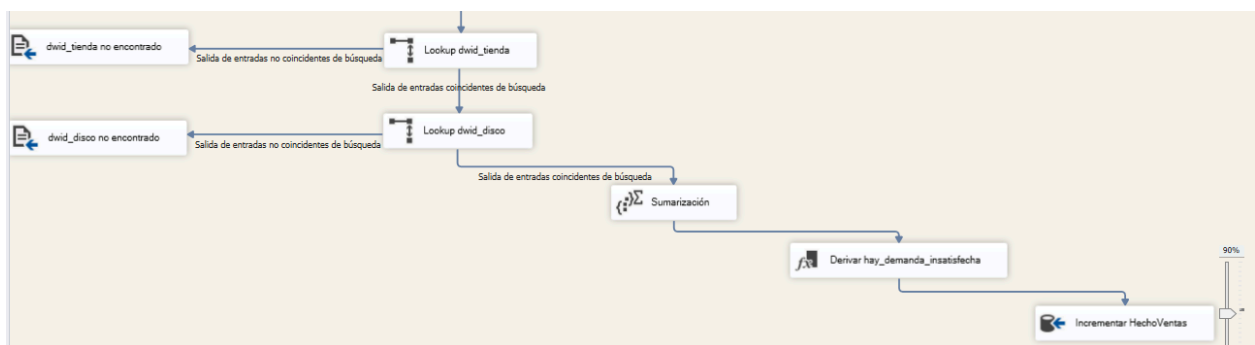


División condicional.

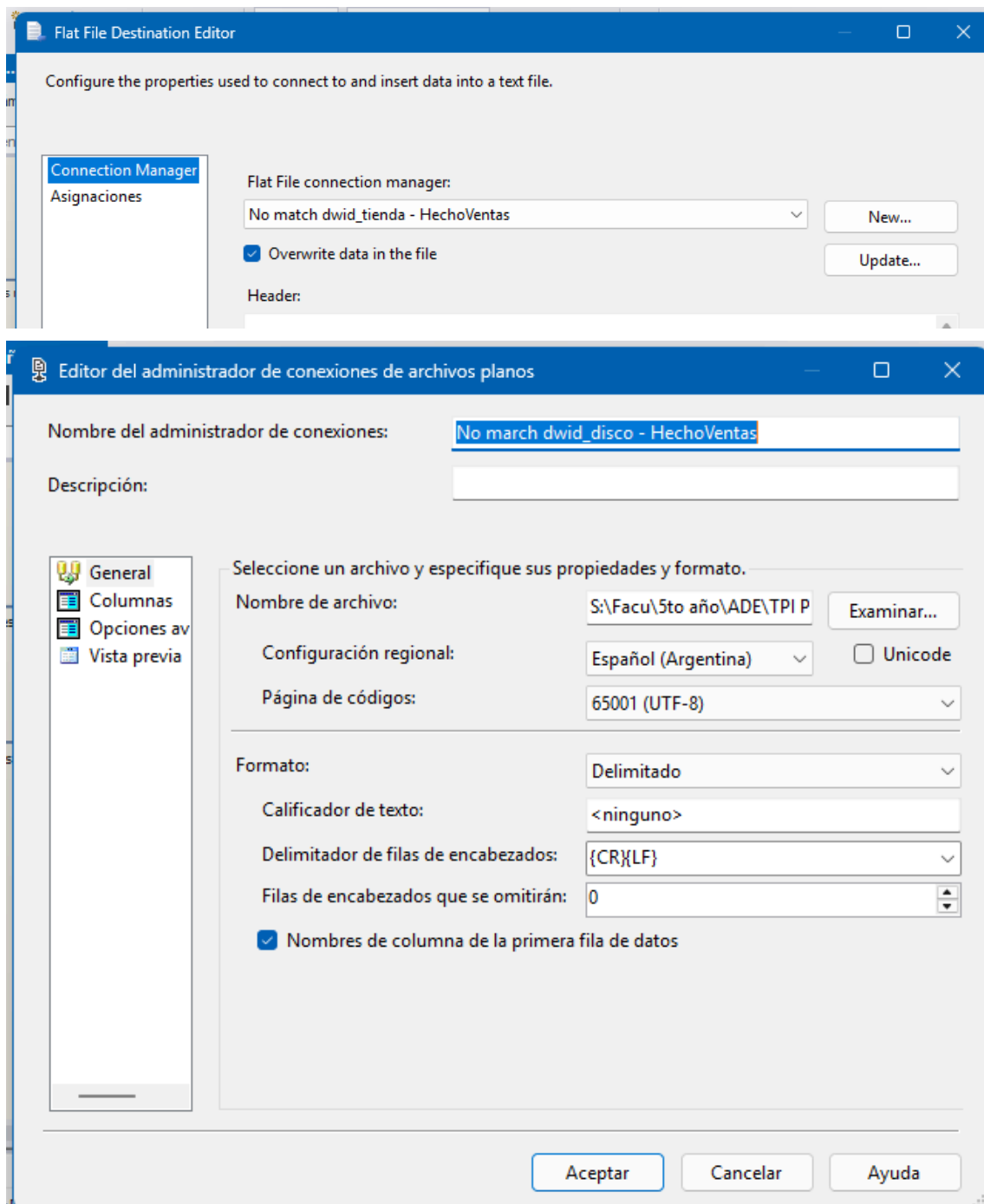
En *División condicional* reflejamos lo que dijimos anteriormente para dividir el flujo para ambos casos. En el Default output name se puede poner cualquier nombre, esta sería una salida en caso de que los rows no encajan con ninguna de las condiciones dadas, lo cual no sucede nunca acá ya que cubrimos todas las posibilidades.

En caso de los NOT NULL entrega hacemos nuevamente la creación de los tiempos como lo hicimos para tiempo_pedido.

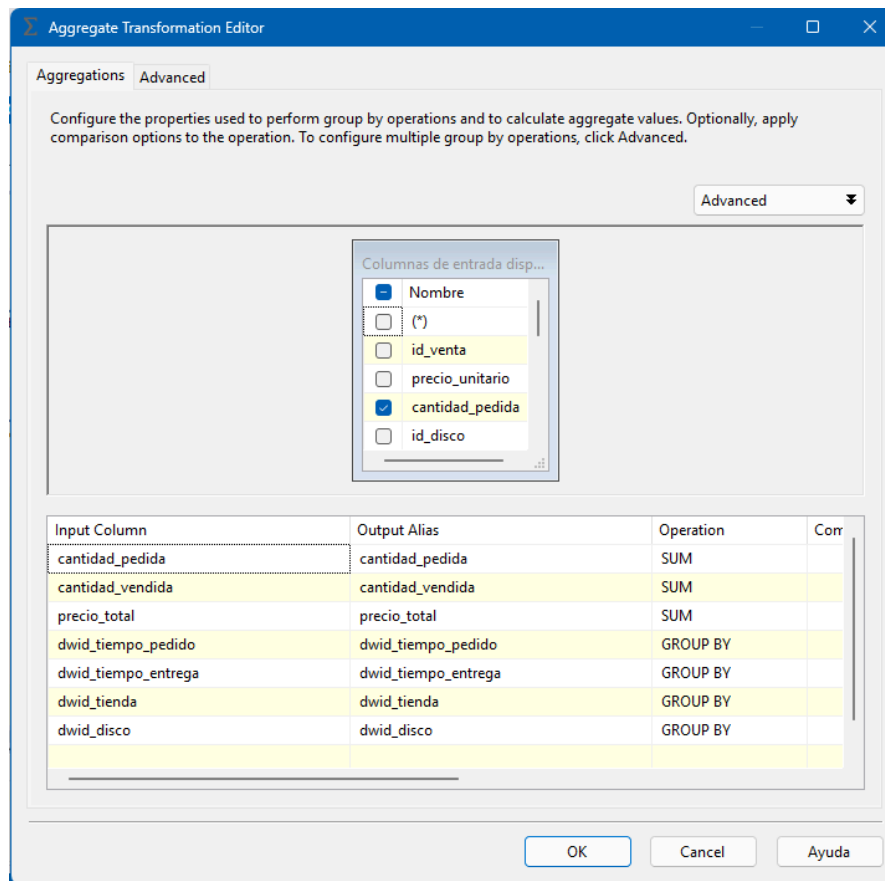
Después volvemos a unir los 3 flujos para seguir con el proceso.



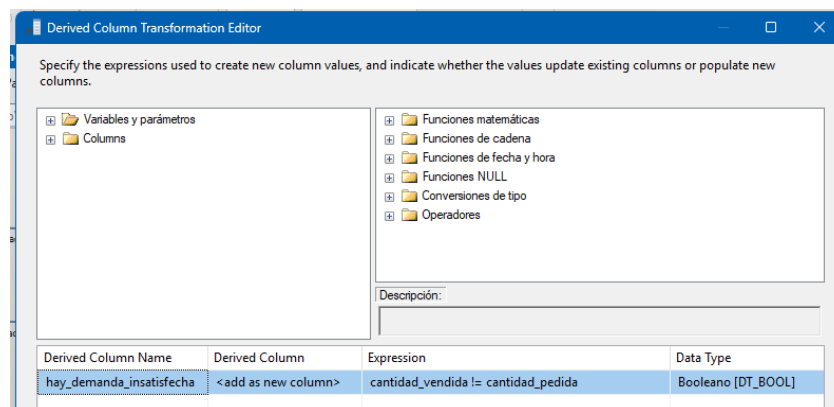
Más adelante, se ejecutan los lookups para obtener los identificadores de las dimensiones restantes: dwid_disco y dwid_tienda. En caso de que alguno de estos lookups no encuentre coincidencias, los registros se derivan a archivos de texto plano con fines de monitoreo y auditoría, por ejemplo: no-match-dwid_disco-HechoVentas.txt.



Para cada uno de estos TXT hace falta una conexión y crear el archivo correspondiente, en este caso de nombre *no-match-dwid_disco-HechoVentas.txt*.



Sumarización.

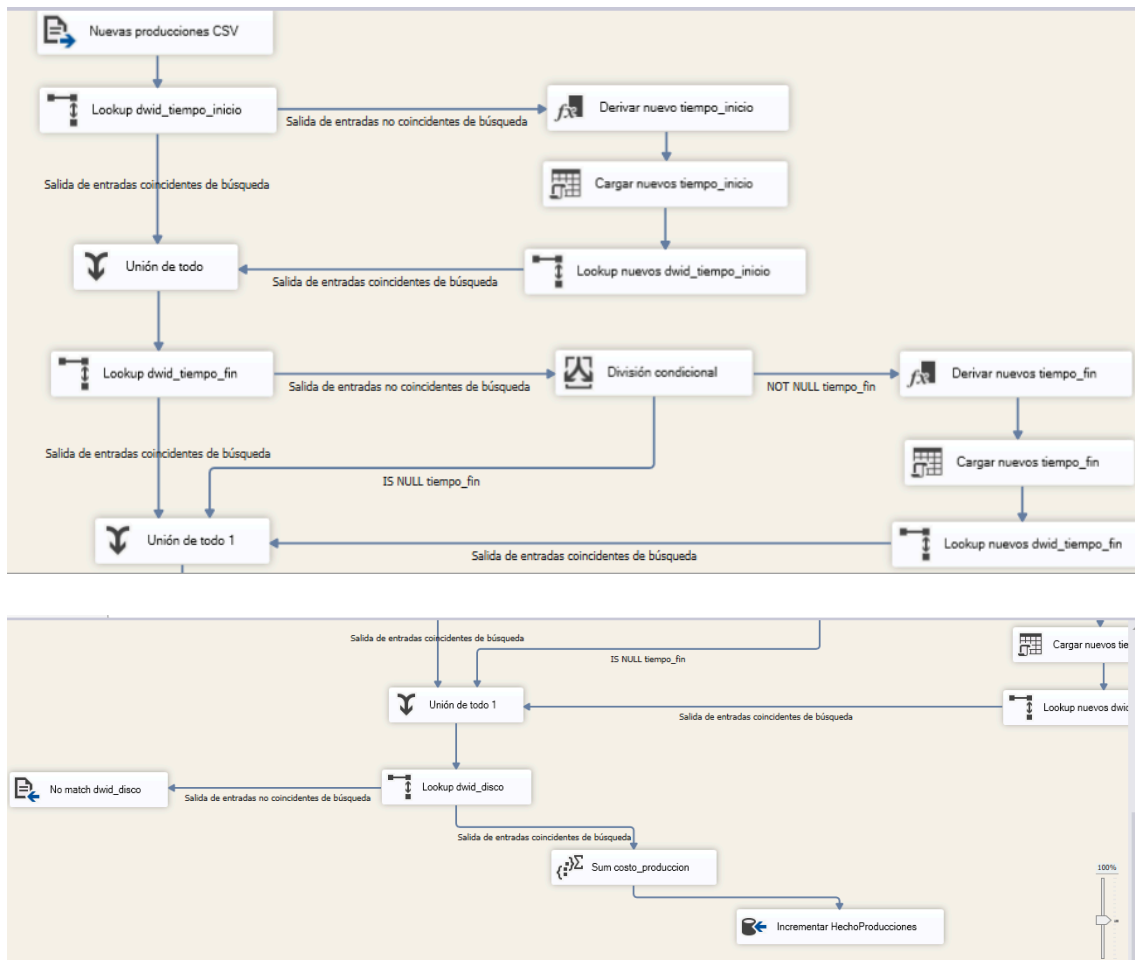


Derivar hay_demanda_insatisfecha.

Una vez resueltas todas las claves foráneas, se realiza la transformación final donde se calcula la columna hay_demanda_insatisfecha, y posteriormente se procede a cargar los datos limpios y completos en la tabla HechoVentas.

ETL Incremento HechoProducciones

El flujo de carga incremental para HechoProducciones sigue una lógica similar a la utilizada en HechoVentas.



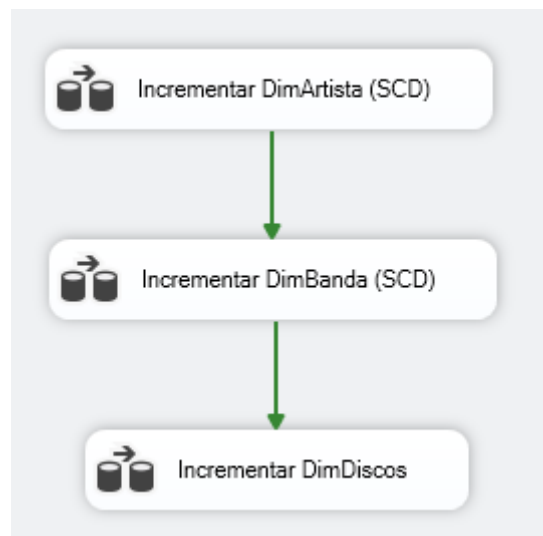
Uno de los aspectos clave de este proceso es el tratamiento de los campos temporales. Al igual que en ventas, se realiza un lookup sobre la dimensión temporal (DimTiempo) para obtener los identificadores correspondientes a las columnas tiempo_inicio y tiempo_fin. En caso de que alguna fecha no se encuentre cargada en DimTiempo, se activa automáticamente una rama del flujo que genera y carga esos nuevos registros, garantizando que no se interrumpa la carga de producciones por falta de fechas.

Para el campo tiempo_fin, se contempla además la posibilidad de que sea NULL, en cuyo caso se utiliza una división condicional que redirige estos casos a un flujo específico que evita la generación innecesaria de fechas. De esta forma, se mantiene la integridad de los datos respetando los valores válidos que provienen del sistema de origen.

Luego de resolver todas las claves temporales, se ejecutan los lookups correspondientes al identificador de disco (dwid_disco). En caso de no encontrar coincidencias, los registros se derivan a archivos de texto para su posterior análisis, tal como se hace en el flujo de ventas.

Por último, se calcula el costo total de producción y se cargan los registros procesados en la tabla HechoProducciones.

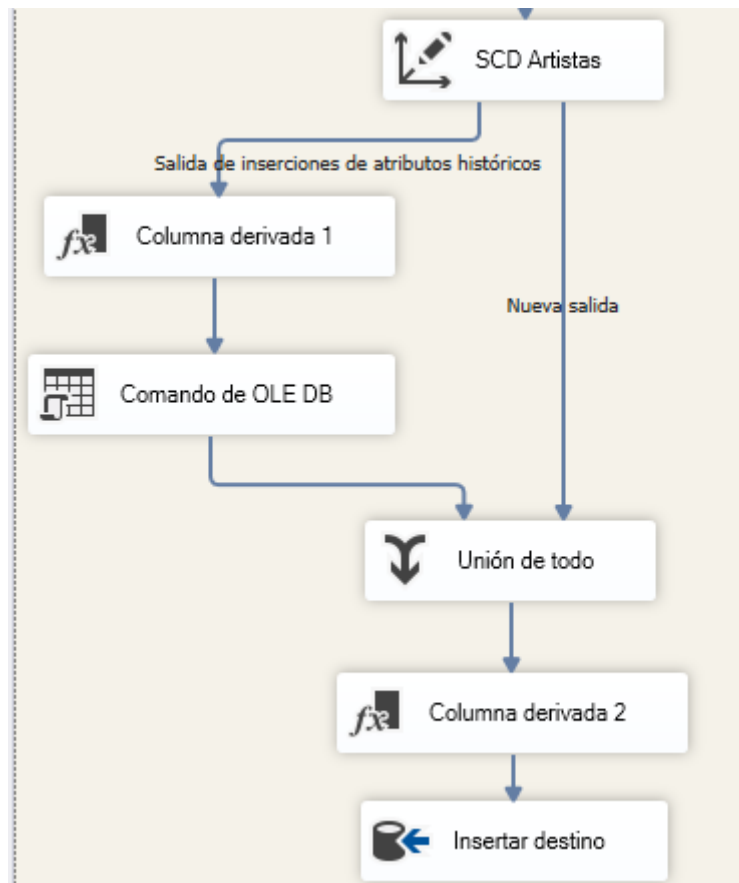
Slowly Changing Dimensions



Incrementar DimArtista (SCD)

En DimArtista aplicamos Slowly Changing Dimensions (SCD) para manejar los cambios históricos en esta, permitiendo conservar versiones anteriores de los registros cuando ocurre una modificación relevante.





Para ir probando el SCD seguimos el camino feliz omitiendo agregar por cada Lookup un archivo de salida TXT donde impactan los No match (o agregar un nuevo instrumento si es la primera vez que viene).

La configuración de SCD que usamos:

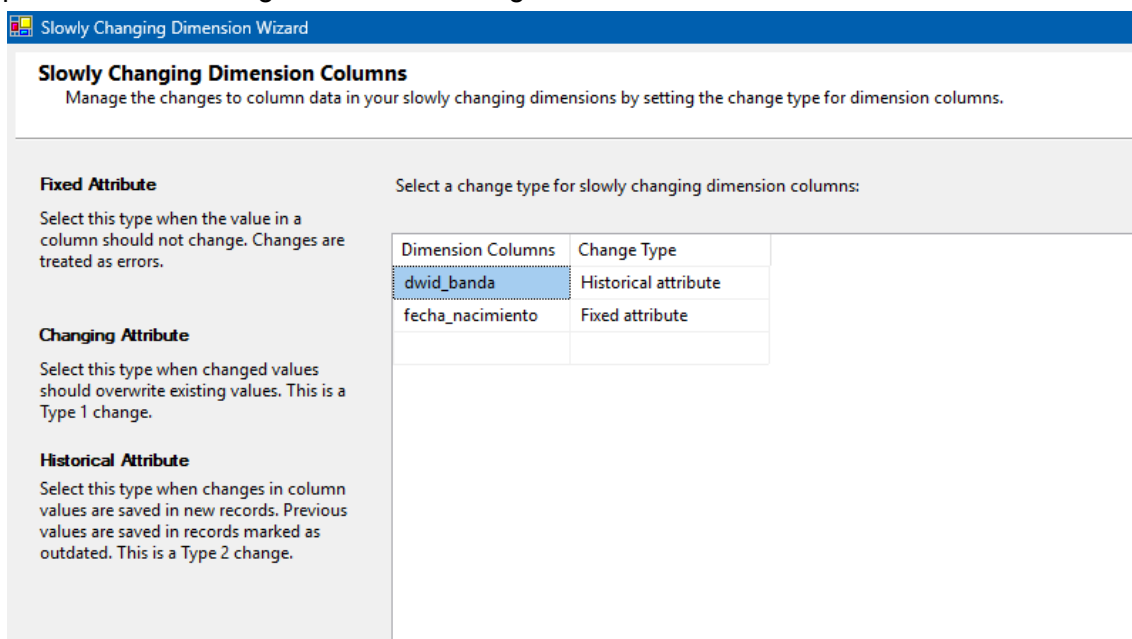
Connection manager:
 CHANO.discografica New...

Table or view:
 [dbo].[DimArtista]

Input Columns	Dimension Columns	Key Type
fecha_nacimi...	fecha_nacimiento	Not a key column
	hasta	
id	id_artista	Not a key column
nombre_artis...	nombre_artistico	Business key
nombre_co...	nombre_completo	Business key
pais_de_origen	pais_de_origen	Not a key column

La implementación se realizó utilizando el asistente de SCD en SSIS. Se definieron como Business Keys los campos nombre_artista y nombre_completo, ya que identifican de forma única a cada artista. Como la fuente de datos (un archivo CSV) no incluye columnas explícitas para las fechas de vigencia (desde y hasta) optamos por omitirlas.

Para probar el SCD configuramos solo las siguientes columnas:

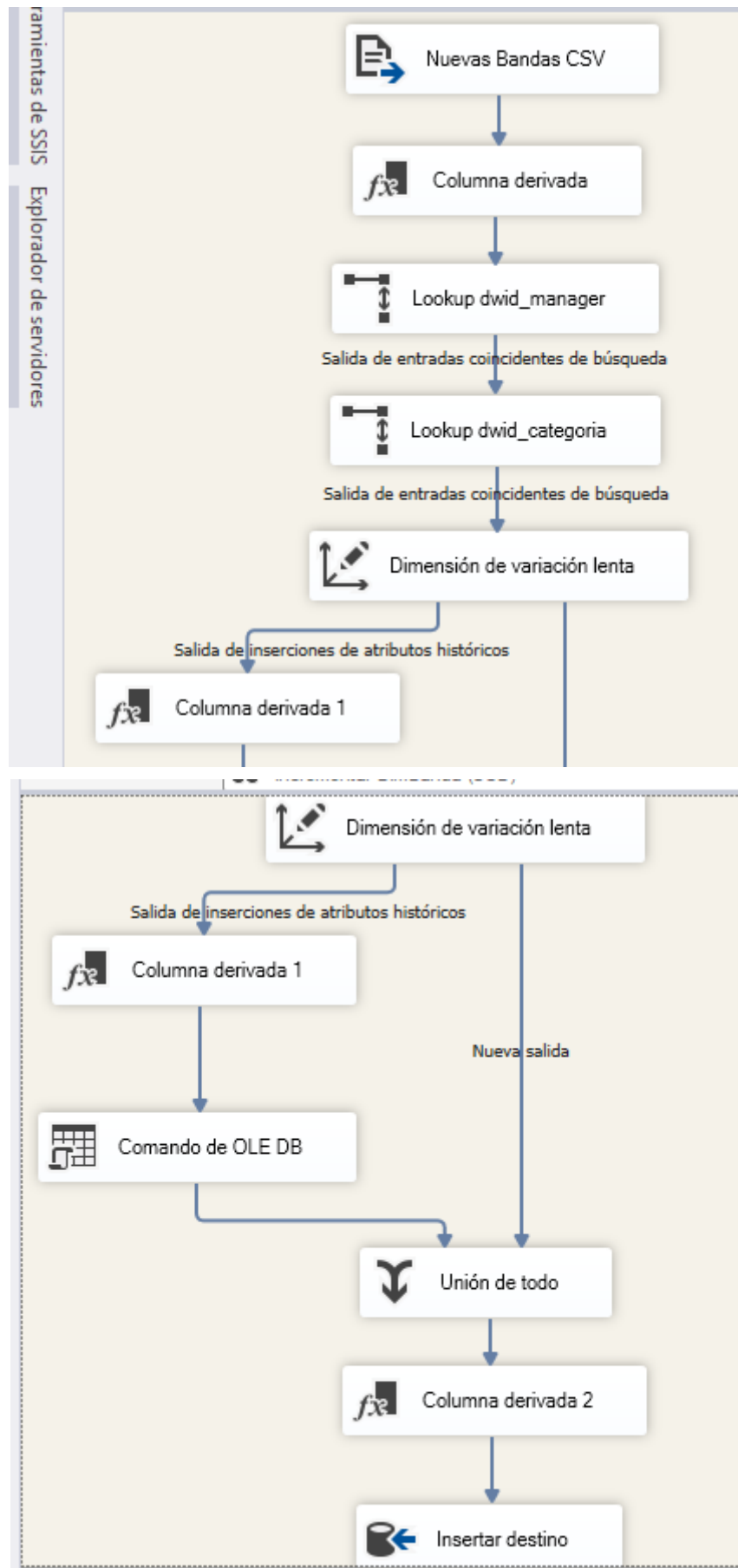


Dimension Columns	Change Type
dwid_banda	Historical attribute
fecha_nacimiento	Fixed attribute

Una vez definida la lógica, el asistente de SSIS se encargó de generar los bloques necesarios para controlar los cambios, comparar versiones y actualizar el Data Warehouse preservando la trazabilidad de cada artista.

Incrementar DimBanda (SCD)

Para implementar el control de cambios históricos en la dimensión DimBanda, fue necesario realizar una modificación estructural en el modelo de datos. Específicamente, se agregaron las columnas desde y hasta a la tabla DimBanda, lo que implicó eliminar y recrear la base de datos del Data Warehouse para reflejar correctamente este ajuste en el esquema. Aunque inicialmente se había considerado aplicar SCD solo a otras dimensiones, se concluyó que tenía más sentido histórico implementarlo en las bandas, por encima incluso de DimManager. Por precaución, las columnas también se mantuvieron en DimManagers, pero su utilización queda sujeta a revisión.



La configuración del flujo SCD se realizó de forma similar a la utilizada en DimArtista, omitiendo en esta etapa la gestión explícita de registros no coincidentes en los lookups. El objetivo principal fue garantizar que los cambios en los atributos de las bandas se registren correctamente, preservando la versión anterior y asignando nuevas fechas de vigencia al nuevo estado.

Una particularidad de esta carga es que se utilizó un CSV adaptado, con el mismo formato que el empleado en la carga inicial, pero agregando una nueva columna manager que contiene el nombre del manager asociado. Esta incorporación fue necesaria para poder realizar el lookup correspondiente, ya que sin esa información no sería posible establecer el vínculo entre la banda y su manager dentro del flujo del SCD.

Configuración del Wizard del SCD:

Select a Dimension Table and Keys
Select a dimension table to load and map columns in the transformation input to columns in the dimension table.

Connection manager:
CHANO.discografica

Table or view:
[dbo].[DimBanda]

Input Columns	Dimension Columns	Key Type
	dwid_representante	
es_activa	es_activa	Not a key column
	hasta	
id	id_banda	Not a key column
name	nombre_banda	Business key
pais_de_origen	pais_de_origen	Not a key column

Help < Back Next > Finish >>| Cancel

Slowly Changing Dimension Wizard

Slowly Changing Dimension Columns
Manage the changes to column data in your slowly changing dimensions by setting the change type for dimension columns.

Fixed Attribute
Select this type when the value in a column should not change. Changes are treated as errors.

Changing Attribute
Select this type when changed values should overwrite existing values. This is a Type 1 change.

Historical Attribute
Select this type when changes in column values are saved in new records. Previous values are saved in records marked as outdated. This is a Type 2 change.

Select a change type for slowly changing dimension col

Dimension Columns	Change Type
dwid_manager	Historical attribute
pais_de_origen	Fixed attribute

Help < Back Next > Finish >>| Cancel

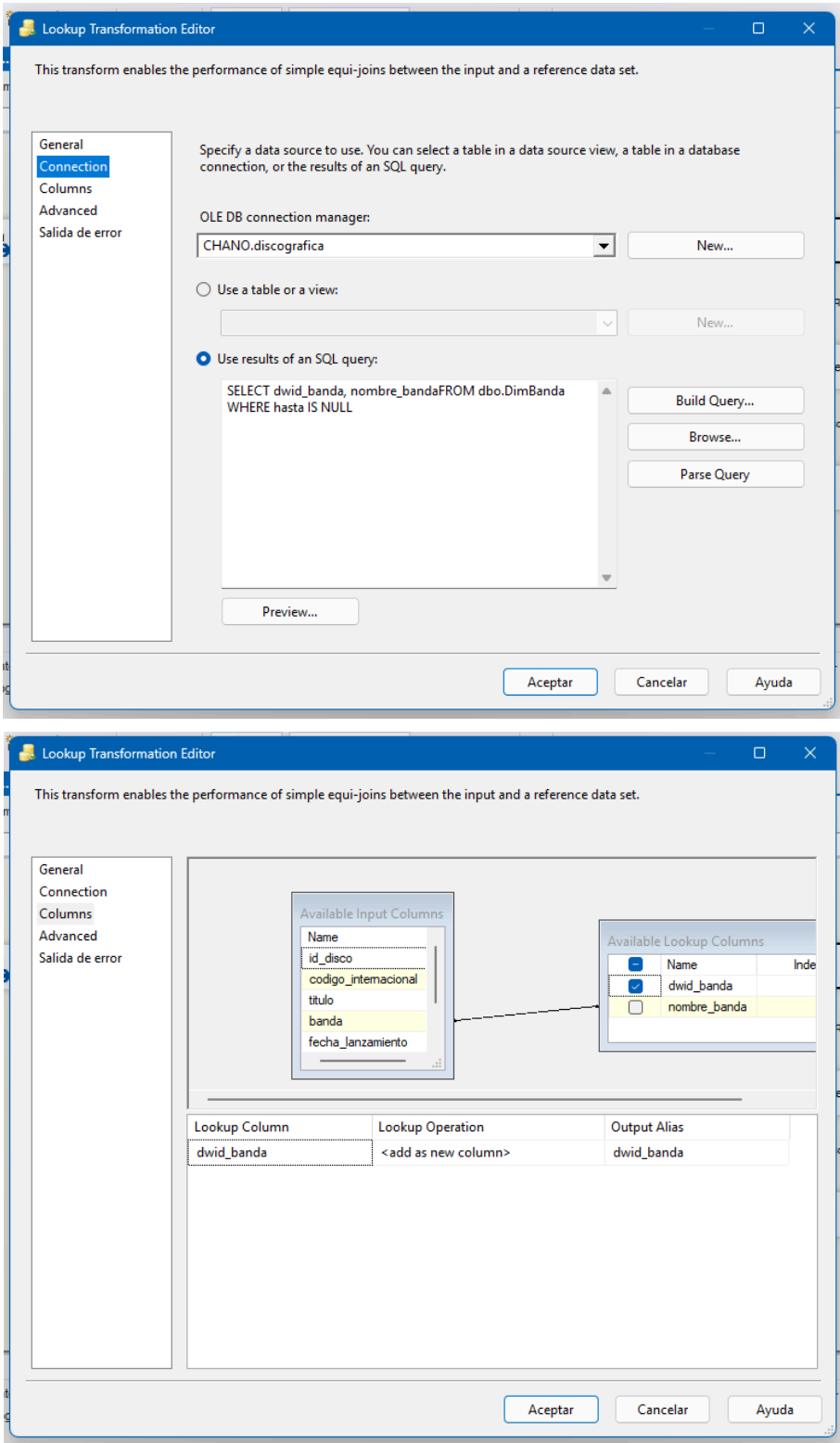
Incrementar DimDiscos



Para comprobar que los nuevos discos que se cargar en DimDiscos cargamos a modo de prueba un par de discos. Estos deberían asociarse a los últimos registros de las Bandas que tienen histórico por el SCD.

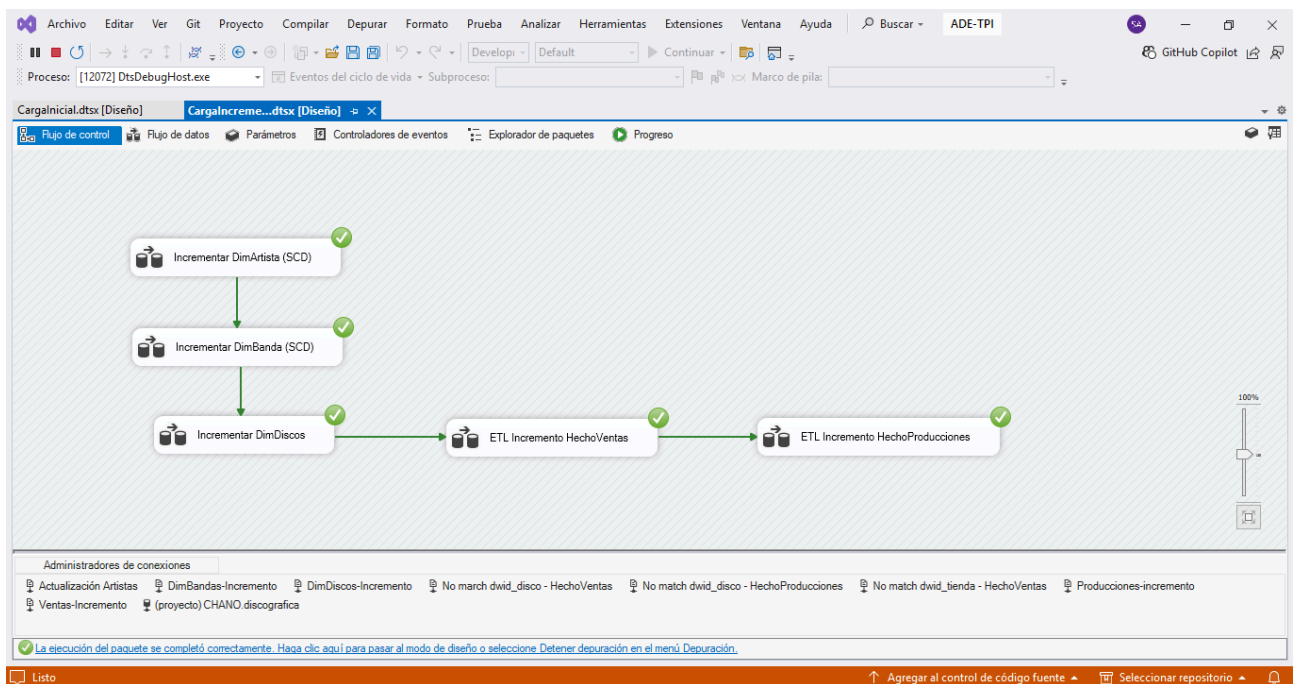
El CSV es del mismo formato al utilizado en la carga inicial, en la derivación completamos todas las columnas necesarias para cargar en la db y en los Lookups recuperamos lo dwid correspondientes (nuevamente todavía no tratamos los no match acá dado que sabemos que no iba a ir por ahí, pero se debería).

Lo destacable acá es que el Lookup dwid_banda trae la primer coincidencia en la db, trayendo así el registro antiguo, sin embargo esto se solventa haciendo el Lookup mediante una query en vez de buscar en la tabla de bandas directamente, en ella le indicamos que solo trabaje con aquellas filas que tenga el campo hasta NULL, ya que estos son los válidos.



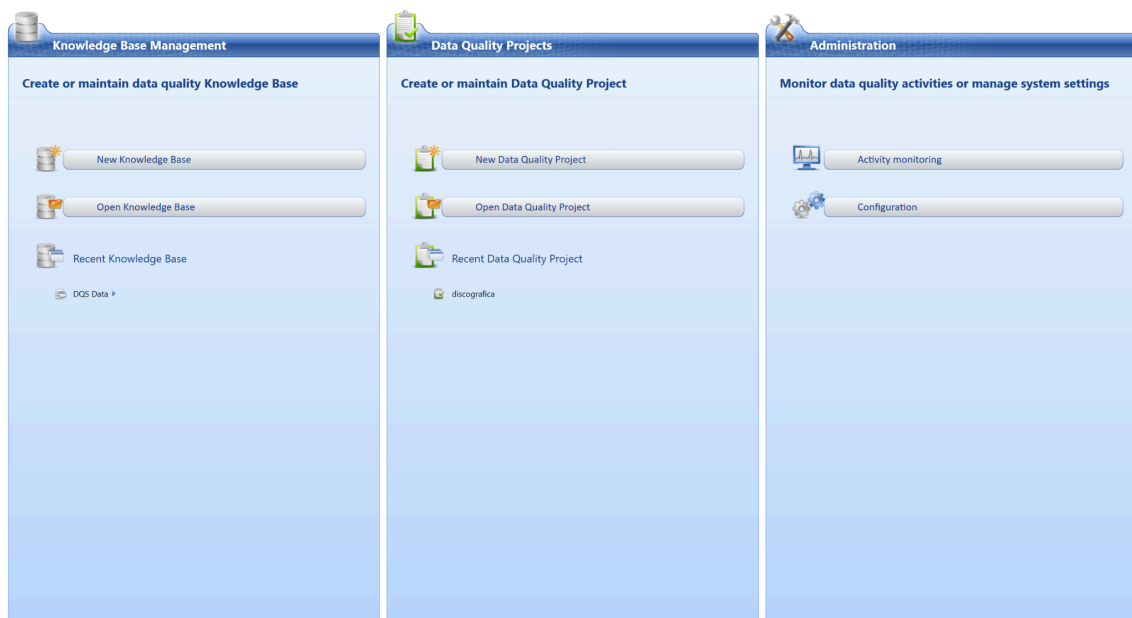
Con ese único detalle a tener en cuenta el disco se asocia a la banda correspondiente.

Ejecución de SCD



Data Quality Services

Primero se ejecuta el SQL Server 2022 Data Services Server Installer. Para eso hubo que habilitar WMIC en Windows 11. Luego, utilizando el SQL Server 2022 Data Services Client:



Pantalla inicial de SQL Server 2022 Data Services Client.

Ya existe una Knowledge Base llamada "DQS Data", la cual tiene un dominio llamado Country/Region. Vamos a utilizar ese dominio para limpiar la columna pais de DimUbicacion.

En Visual Studio en ETL Dimensiones Hoja, agregamos un componente DQS Cleansing.

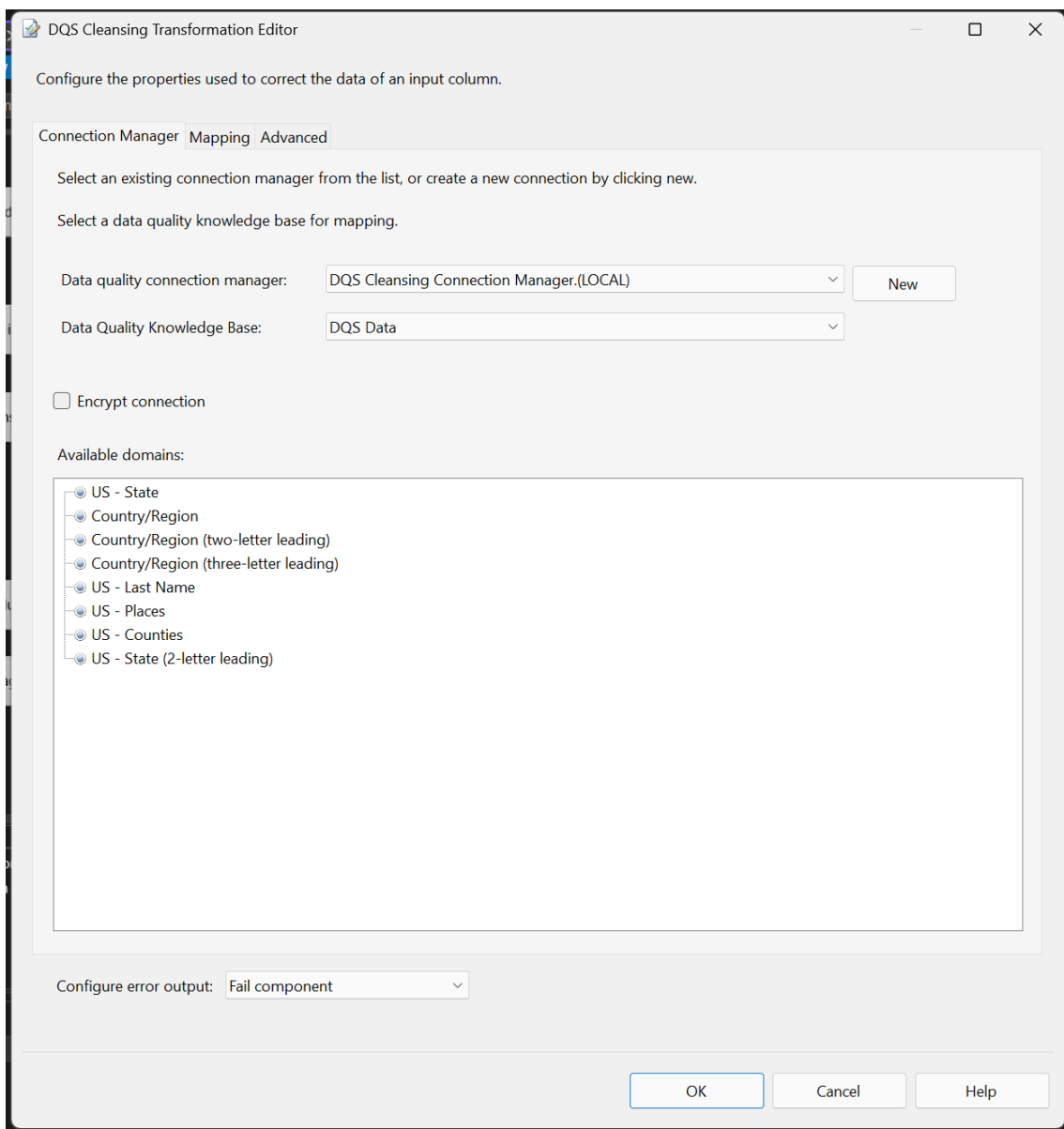


DQS Cleansing en el data flow de DimUbicacion.

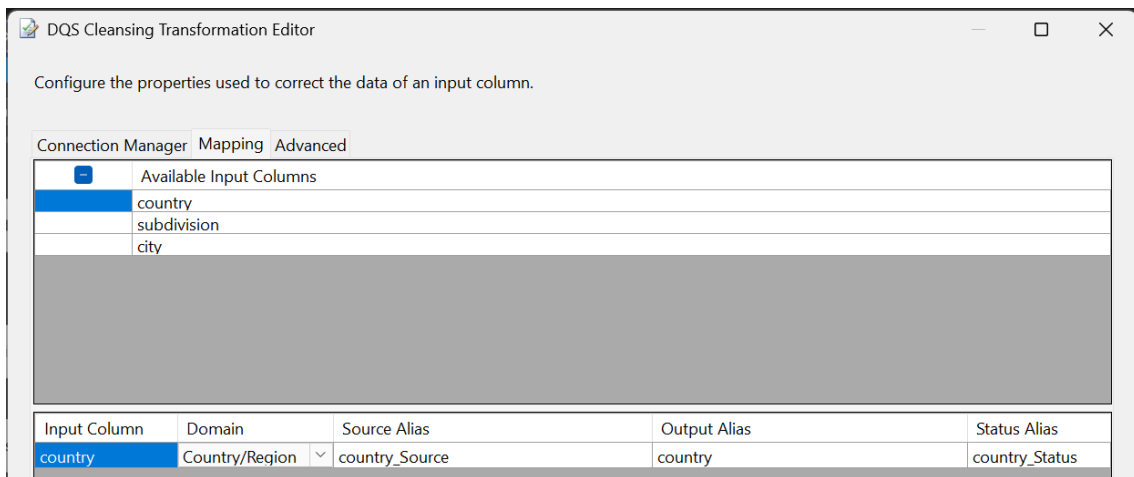
Un problema grande que nos sucedió: si en Project > Properties > Configuration Properties > General > Deployment Target Version > TargetServerVersion el valor es SQL Server 2022, DQS Cleansing me da un error de "Common Language Runtime detected an invalid program."

La solución que encontramos es cambiar esa propiedad a SQL Server 2019 para poder abrir el editor del componente DQS Cleansing y configurarlo. Una vez configurado (según las siguientes dos imágenes), se puede volver a establecer el target como SQL Server 2022.

Esto nos incentivó a dejar de probar las features de DQS, para evitar romper algo del package



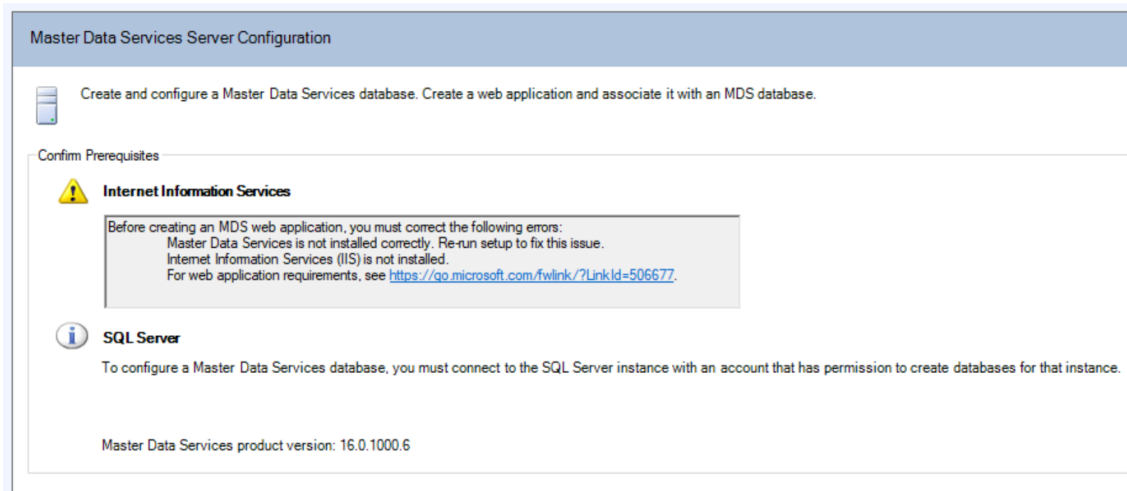
El data quality connection manager fue creado haciendo click en New.



Solo se usa la columna country. El output alias se nombra 'country' para que coincida.

Master Data Services

No fue posible avanzar con Master Data porque para crear una MDS web application eran necesarios ciertos prerequisites. En Windows 11 Home no está presente la feature de Internet Information Services > Security > Windows Authentication que solicita el mensaje mostrado más abajo. Por ende, no es posible levantar el cliente web para interactuar con el servicio de Master Data, a menos que se cuente con una versión Windows 11 Pro. Debido a que ningún miembro contaba con esta versión de Windows, se decidió no implementar MDS en el proyecto.



Mensaje de prerequisites en el Master Data Configuration Manager.